Beyond Fidelity: Explaining Vulnerability Localization of Learning-Based Detectors

BAIJUN CHENG, Peking University, Beijing, China
SHENGMING ZHAO, University of Alberta, Edmonton, Canada
KAILONG WANG and MEIZHEN WANG, Huazhong University of Science and Technology, Wuhan, China
GUANGDONG BAI, University of Queensland, Saint Lucia, Australia
RUITAO FENG, University of New South Wales, Sydney, Australia
YAO GUO, Peking University, Beijing, China
LEI MA, University of Alberta, Edmonton, Canada
HAOYU WANG, Huazhong University of Science and Technology, Wuhan, China

Vulnerability detectors based on deep learning (DL) models have proven their effectiveness in recent years. However, the shroud of opacity surrounding the decision-making process of these detectors makes it difficult for security analysts to comprehend. To address this, various explanation approaches have been proposed to explain the predictions by highlighting important features, which have been demonstrated effective in domains such as computer vision and natural language processing. Unfortunately, there is still a lack of in-depth evaluation of vulnerability-critical features, such as fine-grained vulnerability-related code lines, learned and understood by these explanation approaches. In this study, we first evaluate the performance of ten explanation approaches for vulnerability detectors based on graph and sequence representations, measured by two quantitative metrics including fidelity and vulnerability line coverage rate. Our results show that fidelity alone is insufficent for evaluating these approaches, as fidelity incurs significant fluctuations across different datasets and detectors. We subsequently check the precision of the vulnerability-related code lines reported by the explanation approaches, and find poor accuracy in this task among all of them. This can be attributed to the inefficiency of explainers in selecting important features and the presence of irrelevant artifacts learned by DL-based detectors.

CCS Concepts: • **Security and privacy → Software security engineering;**

Additional Key Words and Phrases: Vulnerability Detection, Explanation Approaches, Fidelity, Coverage Rate

B. Cheng and M. Zhao equal contribution.

Authors' addresses: B. Cheng and Y. Guo (Corresponding author), Peking University, 5 Yiheyuan Road, Haidian District, Beijing, China, 100871; e-mails: prophecheng@stu.pku.edu.cn, yaoguo@pku.edu.cn; S. Zhao and L. Ma, University of Alberta, 11011-88 Avenue, Edmonton, Alberta, Canada, T6G 2G; e-mails: shengmi1@ualberta.ca, ma.lei@acm.org; K. Wang (Corresponding author), M. Wang, and H. Wang (Corresponding author), Huazhong University of Science and Technology, 1037 Luoyu Road, Hongshan District, Wuhan, Hubei, China, 430074; e-mails: wangkl@hust.edu.cn, mzwang@hust.edu.cn, haoyuwang@hust.edu.cn; G. Bai, University of Queensland, St Lucia QLD, Brisbane, Queensland, Australia, 4072; e-mail: g.bai@uq.edu.au; R. Feng, University of New South Wales, High Street, Sydney, New South Whales, Australia, 2052; e-mail: ruitao.feng@unsw.edu.au.

1 INTRODUCTION

The ubiquity of software in today's digital society underpins the paramount importance of vulnerability detection techniques. As we propel further into the digital age, software permeates every corner of our lives, from healthcare to finance, transportation to communication. This pervasive deployment has led to an exponential increase in the surface area for potential attacks, amplifying the demand for robust, accurate, and efficient vulnerability detection mechanisms. Current vulnerability detection techniques [1–15], however, are often characterized by significant limitations that stymie their effectiveness. A core shortcoming lies in their limited ability to pinpoint the exact code lines that instigate vulnerabilities. Existing detection tools frequently flag entire segments of code as potentially hazardous, leading to vast sections needing review. This wide-net approach not only overwhelms developers with high rates of false positives but also sows the seeds of inefficiency and can potentially delay mitigation efforts. The ability to identify the precise lines of code responsible for a vulnerability, thus, is an issue of critical importance and poses an intricate challenge in the realm of program analysis.

In addressing the challenge, researchers have turned to the promising realm of explainable AI techniques. These methods illuminate the defining features within detected vulnerabilities, thereby equipping developers with the insights necessary to comprehend and rectify these issues more effectively. The essence of these techniques is in highlighting the key characteristics of vulnerable programs, playing a critical role in localizing vulnerability-correlated code statements. Such explanation techniques have been successfully integrated with **Deep Learning** (**DL**)-based detectors in the vulnerability detection area. For instance, Li et al. [10] incorporated the GNNExplainer [16] into their proposed detector, while Wu et al. [14] employed Grad-CAM++ [17] in their image-based vulnerability detector, VulCNN. In another innovative approach, Zou et al. [18] designed a heuristic search algorithm to identify pivotal tokens within code gadgets, generated by VulDeepecker [13] and SySeVR [12]. It is noteworthy that these explanation techniques are not confined to the field of vulnerability detection; they have found valuable applications in a variety of domains, including **computer vision** (**CV**) [19] and **natural language processing** (**NLP**) [20], primarily in the context of DL models used for vulnerability detection.

Despite the evident potential of existing explanation techniques, our observations have revealed a disconnect between the explanation results and their intrinsic relevance to the identified vulnerabilities. In some instances, the explainer models mistakenly allocate high relevance scores to code statements that bear no relation to the underlying vulnerabilities. This observation incited us to undertake a thorough performance analysis of these explanation techniques within the specific scope of vulnerability detection.

In our study, *the overarching objective is to conduct a rigorous evaluation and examination of the performance of these explanation techniques within the field of fine-grained vulnerability localization.* We set our sights on evaluating five graph-based explanation methodologies: GNNExplainer [16], PGExplainer [21], GNN-LRP [22], GradCAM [23], and DeepLift [24]. These techniques are examined in conjunction with four graph-based vulnerability detectors, namely DeepWuKong [11], IVDetect [10], Reveal [9], and Devign [8]. Furthermore, we venture into the investigation of five sequence-based techniquesLRP [25], Shap [26], Lime [27], GradInput [23],

and DeepLift [24]in relation to sequence-based vulnerability detectors such as VulDeePecker [13], SySeVR [12], and TokenLSTM.

In our evaluation process, we initially use fidelity as the metric to evaluate existing explanation techniques, a common practice in explainable AI to assess the accuracy of generated explanations. However, we find that relying solely on fidelity leads to significant fluctuations in scores across different detectors and datasets, showing inconsistent results. To address this, we introduce an additional metric called Vulnerability **Line Coverage** (**LC**) rate, mirroring MSR [28], which can be divided into two sub-metrics: the Vulnerability-**Triggering Code Line Coverage** (**TLC**) rate and the **Vulnerability-Fixing Code Line Coverage** (**FLC**) rate. These aim at providing a more detailed understanding of how well the explanation techniques can identify and explain vulnerabilities. However, our findings show that most explanation techniques struggle to pinpoint key vulnerability-related statements accurately, as seen in both TLC and FLC measurements (RQ2, Section 5.4). For instance, when applying GNNExplainer to IVDetect on the SARD dataset, the results show a wide range of TLC values from 7% to 64%, and FLC values from 18% to 66%, despite the fidelity scores remaining relatively high between 74% and 92%. This highlights that existing approaches, even with high fidelity scores, fail to learn and reflect the critical features inherent to the vulnerabilities, indicating that a broader evaluation beyond fidelity is necessary to truly understand the effectiveness of explanation techniques in addressing vulnerabilities.

Throughout our study, we have gained two in-depth understandings regarding the DL-based vulnerability detectors and explanation approaches, respectively. On the one hand, the existing explanation approaches are prone to pick up vulnerability-irrelevant features as explanations. On the other hand, DL-based detectors indeed perform vulnerability detection by diligently learning the distinctions between vulnerable and normal samples. The varying sensitivities of model-related explainers on retrained and previous models to vulnerability-fixing lines suggest that they may primarily identify feature discrepancies between positive and negative samples, rather than understanding how vulnerabilities are triggered (RQ3.1, Section 5.5.1). To further validate our point and evaluate their comprehension of vulnerabilities, we manually patch a subset of vulnerabilities, using the originally patched samples as a reference. By comparing detection accuracy between non-vulnerability samples, we anticipate minimal accuracy differences if the detectors truly understand vulnerability nuances (RQ 3.2, Section 5.5.2). Surprisingly, a significant portion of manually fixed codes is misclassified as vulnerable, with fewer false alarms in the originally patched samples. Moreover, we identify a kind of irrelevant text features that notably influence the model's predictions, collectively indicating that current detectors have not fully understood vulnerability semantics.

Our main contributions could be summarized as follows:

- **Enhancing the evaluation metrics for explanation approaches.** We employ the quantitative metric of fidelity and introduce additional metrics, namely TLC and FLC, to evaluate the efficacy of the explanation approaches in accurately localizing vulnerabilities. Our investigation reveals that none of the explanation approaches effectively localize vulnerabilities even when they achieve high fidelity scores.
- **Proposing a new evaluation metric for DL-based detectors.** To complement the existing evaluation metrics, we introduce a new metric VUR that is focused on the impact of vulnerability-unrelated perturbations on DL-based detectors. The metric is employed to determine the proportion of perturbations unrelated to vulnerabilities. We observe that all detectors are affected to varying extents by vulnerability-unrelated perturbations, potentially causing a flip in their prediction outcomes.
- **Revealing the limitations in DL-based detectors.** As deep-learning vulnerability detectors aim to identify vulnerabilities by learning distinctions between vulnerable and

normal samples, we find that the models can still be misdirected by irrelevant disturbances. Furthermore, these models are unable to recognize vulnerability-fixing patterns not present in the training dataset. We also identify a specific text feature that can trigger false positives, indicating DL-based detectors primarily distinguish between vulnerable and normal samples without a full comprehension of the mechanisms underlying vulnerabilities.

To facilitate the reproductivity of our analysis, we open-source our codes and datasets. The website is https://github.com/for-just-we/VulDetectArtifact.

2 BACKGROUND

In this section, we primarily introduce the DL-based detectors and explanation approaches used in this study.

2.1 DL-Based Vulnerability Detectors

In the realm of vulnerability detection, DL-based approaches have emerged as potent tools for identifying security vulnerabilities within software programs. These methodologies typically entail segmenting source code into discrete code fragments, subsequently vectorizing each fragment using DL models, and predicting the likelihood of a vulnerability existing within that fragment. The inherent advantage of these methods lies in their capacity to autonomously discern patterns and regularities within source code, thereby enhancing the accuracy and efficiency of vulnerability detection.

Based on how source code is segmented, DL-based vulnerability detectors can be categorized into two primary classes: function-level and slice-level detectors. In function-level detectors [8–10, 15, 29–31], individual functions within the program are treated as isolated code fragments. In contrast, slice-level detectors [11–13, 32, 33] extract code fragments spanning multiple functions based on data dependencies or control flow dependencies of vulnerability-critical statements. This approach offers a comprehensive contextual view of vulnerabilities, thus contributing to improved detection precision.

Furthermore, concerning the vectorization of code fragments, these vulnerability detectors can be further classified as sequence-based and graph-based detectors. Sequence-based methods directly parse source code fragments into token sequences and employ sequence neural networks for vectorization. Conversely, graph-based approaches typically parse the structures of code fragments, such as **abstract syntax trees** (**AST**), **control flow graphs** (**CFGs**), or **program dependency graphs** (**PDGs**), and further process these graph structures using graph neural networks. This facilitates the capture of structural information and dependency relationships within the source code, thereby enhancing the effectiveness of vulnerability detection.

Our study involves seven DL-based detectors: DeepWuKong [11], Reveal [9], Devign [8], IVDetect [10], TokenLSTM, VulDeePecker [13], SySeVR [12]. Table 1 provides an overview of these detectors and their key characteristics. Specifically:

— Devign extracts a comprehensive joint graph that encompasses the AST, CFG, **Data Dependence Graph** (**DDG**), and **Natural Code Sequence** (**NCS**) for each function. It generates initial embeddings for each node with Word2Vec [34]. Also, its DL model is composed of a gated graph recurrent layer (GGRN) [35], a convolutional layer, and an MLP Layer.
— Reveal follows a similar pipeline as Devign. However, in comparison to Devign, Reveal represents each function using a **code property graph** (**CPG**) [36], which is constructed from the AST, CFG, and **Program Dependence Graph** (**PDG**). Additionally, when aggregating all node embeddings into a graph embedding, Reveal employs a summation approach for all

Table 1. Overview of Seven DL-based Detectors

Approaches	Representation	Granularity	Model
TokenLSTM	token sequence [43]	function [43]	BLSTM [41]
VulDeePecker [13]	token sequence	slicing [43] + DDG [36]	Word2Vec [34] + BLSTM
SySeVR [12]	token sequence	slicing + PDG [36]	Word2Vec + BGRU [38]
DeepWuKong [11]	PDG	slicing + PDG	Doc2Vec [40] + GCN [39]
Devign [11]	CFG [36] + DDG + AST + NCS	function	Word2Vec + GGRN [35]
Reveal [9]	CPG [36]	function	Word2Vec + GGRN
IVDetect [10]	PDG	function	Glove + TreeLSTM [44] + GCN

node embeddings instead of using a convolutional layer. Reveal employs a similar pipeline with Devign.

— IVDetect extracts PDGs for each function and generates initial embeddings using sub-token sequences, variables, and types, as well as their surrounding contexts, by leveraging Glove [37] and GRU [38]. Subsequently, it employs a **graph convolutional network** (**GCN**) model [39] to update node embeddings and aggregates them using a summation approach. To identify vulnerabilities at the statement-level, IVDetect selects crucial sub-graphs within each PDG using the GNNExplainer. [16].

— DeepWuKong, like IVDetect, extracts PDGs to identify vulnerabilities. However, it differs in its approach. DeepWuKong begins by inter-procedurally slicing a program, specifically focusing on statements that could potentially trigger vulnerabilities. Subsequently, it embeds each sliced PDG using a combination of Doc2Vec [40] and GCN.

— TokenLSTM straightforwardly parses each function into a token sequence and utilizes a Bi-LSTM [41] to determine if the function might contain vulnerabilities.

— VulDeePecker initiates the detection process by first identifying statements that involve vulnerability-related system API calls and subsequently slices the program accordingly using DDG. It then proceeds to vectorize and make predictions for each slice using Word2Vec [34] and Bi-LSTM [41].

— SySeVR expands upon VulDeePecker's approach by broadening the set of vulnerability-related statements and employing PDGs for slicing. Additionally, it substitutes the use of Bi-LSTM with Bi-GRU [38] for the purpose of vulnerability detection.

All of these DL-based detectors approach vulnerability detection as a binary classification problem, aiming to determine whether a given code fragment contains vulnerabilities. Initially, the code fragment is processed into a feature vector, which is then fed into a classifier to predict the probability of the code fragment containing vulnerabilities. It is worth mentioning that the TokenLSTM method is an implementation based on the detector proposed by Russel et al. [15]. To ensure consistency in our experiment settings, we implement the TokenLSTM method utilizing the **bidirectional LSTM** (**BLSTM**) [41] model as the feature extraction layer, while excluding the random forest classifier [42] from the architecture.

2.2 Explanation Approaches

In recent years, there has been notable progress in the field of DL, particularly in its application to vulnerability discovery. However, it's important to recognize that these DL models typically excel in binary classification tasks. Despite their success, their decision-making processes remain largely concealed within black-box models, which can raise trust and transparency concerns.

This lack of transparency and interpretability has raised skepticism regarding their use in real-world security applications. Fortunately, explainability methods have emerged as a promising approach to address this issue. These methods offer a fresh perspective by shedding light

Table 2. Basic Information of Explanation Approaches

Explanation Method	Representation	Introduction
GNNExplainer [16]	Graph	GNNExplainer trains mask for edges and node features to identify meaningful input information. These masks are updated to maximize the mutual information between original and updated predictions.
PGExplainer [21]	Graph	PGExplainer trains a parameterized mask predictor to predict the edge mask value. The predictor takes embedding vectors of two nodes connected by an edge as input and produces a corresponding mask value for the edge.
GNN-LRP [22]	Graph	GNN-LRP explores the importance of various graph walks through score decomposition. It decomposes model predictions into different graph walks using LRP
GradCAM [23]	Graph	Pope et al. [23] extend GradCAM [46] to the realm of graph classification, where it initially calculates the gradients of the target prediction with respect to the final node embeddings. Subsequently, these gradients are averaged to derive the weights for each feature map.
DeepLift [24]	Both	DeepLift combines LRP and Integrate Gradient, utilizing their insights to enhance its backpropagation process. By incorporating a reference input, DeepLift overcomes discontinuities and achieves superior performance in various tasks compared to traditional LRP methods. It is extended to support deep graph models by Yuan et al. [45].
Lime [27]	Sequence	Lime employ a surrogate model to approximate the performance of the target model. By perturbing the input x, LIME samples neighboring points $x_1^{'}, x_2^{'}, \ldots, x_n^{'}$ and obtains their labels through the target model.
Shap [26]	Sequence	Shap estimates the Shapley values of each feature in the prediction by adjusting the loss function, weight kernel, and solving a regression problem, thus providing additive explanations.
Gradient * Input [23]	Sequence	Gradient * Input is an early-stage gradient-based attribution method that computes attribution by taking the partial derivative of the output with respect to the input and multiplying it by the input itself.
LRP [25]	Sequence	LRP is a technique that approximates feature importance scores by leveraging the internal relationship between input and output in neural networks. It uses backpropagation to propagate relevance scores from the output layer to the input layer, assigning importance values to each input feature.

on the inner workings of machine learning systems. It's essential to acknowledge that their implementation often requires considering the context outlined earlier.

We overview the explanation approaches employed in this study, categorized into the following two classes based on the type of target detectors. Table 2 provides an overview of the explanation approaches employed in this study. It is worth noting that in our research, the explanation approach DeepLift [24] offers both graph-based and sequence-based versions, we consider them as distinct interpretation methods. Specifically:

— **LIME** [27] fits a surrogate model for every sample x in the sample space to approximate the performance of target model $f(x)$, and the surrogate model is self-explainable. To fit a surrogate model, LIME samples a number of points through perturbing x, then preserves a set of perturbations $x_1^{'}, x_2^{'}, \ldots, x_n^{'}$ near x in the sample space. After that, LIME utilizes the

target model to gain the label of each perturbation and uses them as the train set to fix a linear model, which has a similar decision boundary to the target model.

— **SHAP** [26] approximates the Shapley value of each feature in the sample concerning the final prediction. This approach is inspired by the foundation of LIME and involves redefining the loss function $\mathcal{L}$ and the weight kernel $\pi_x(x_i^{'})$ based on the Shapley value principle. The Shapley additive explanation values are computed through a regression process.

— **Gradient-based** [23] explanation methods quantify the impact of a feature, denoted as x_i, in the input sample on the model's output, o, by calculating the partial derivative of o concerning feature i. In this research, we opt for $grad_{input}$, which represents a gradient-based method and is calculated as the dot product of the input vector and the partial gradient.

— **LRP** (i.e., layer-wise relevance propagation) represents another significant approach for leveraging the internal relationships within neural networks to estimate feature importance scores. In essence, LRP utilizes the backpropagation mechanism of neural networks to propagate the output scores (relevance) from the model's output layer all the way back to the input layer. Consequently, each input feature receives a relevance value, serving as an importance score. To apply the LRP method to LSTMs and GRUs, we employ the propagation rule introduced by Arras et al. [25].

— **DeepLift** [24] amalgamates the principles of LRP and Integrated Gradients. Much like LRP, DeepLift utilizes a backward propagation process. However, it introduces the notion of reference input from Integrated Gradients. This inclusion helps mitigate discontinuities and yields enhanced performance across various tasks compared with standard LRP approaches. Notably, DeepLift has been extended to support graph models by Yuan et al. [45].

— **GNNExplainer** [16] employs a dual approach for learning importance in the input data, incorporating soft masks for edges and hard masks for node features. These masks are initially initialized with random values and iteratively updated to maximize the mutual information between the original predictions and the new predictions. It's worth noting that, in this context, the node features mask is omitted due to the fact that each node is embedded with a low-dimensional word vector, and individual dimensions within the feature vector do not carry any specific semantic significance.

— **PGExplainer** [21] generates discrete masks for edges as explanation results. Differing from GNNExplainer, it employs a parameterized mask predictor to estimate the edge mask values. This predictor takes the embedding vectors of the two nodes linked by an edge as input and computes the associated mask value for that edge.

— **Grad-CAM** [23] is a widely adopted explanation model for image classifiers. It has been extended for use with graph models to assess the significance of various nodes. The fundamental concept revolves around the fusion of hidden feature maps and gradients to determine node importance.

— **GNN-LRP** [22] investigates the significance of various graph walks through score decomposition. This approach aligns well with deep graph neural networks, as these walks correspond to message flows during neighborhood information aggregation. GNN-LRP leverages high-order Taylor decomposition of model predictions to break down the ultimate predictions into distinct graph walks using the LRP technique.

3 MOTIVATING EXAMPLE

Figure 1(a) presents the function "parserep" in the tcpdump project, designed for parsing reply packets. This code is identified as vulnerable in **Common Vulnerabilities and Exposures** (**CVE**-2017-12898) listed in the **National Vulnerability Database** (**NVD**). The associated commit log for

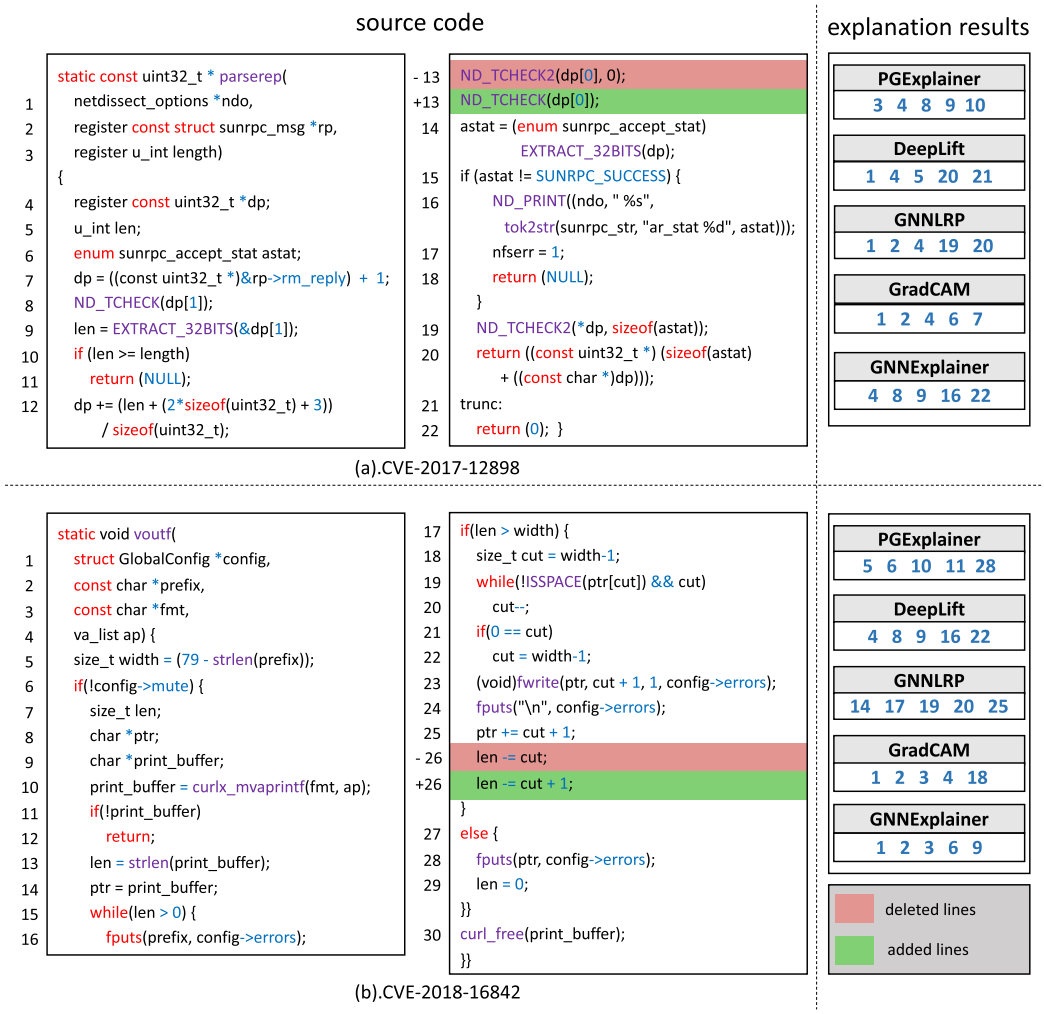


Fig. 1. An illustrative example featured by CVE-2017-12898 and CVE-2018-16842. In CVE-2017-12898, a modification in line 13 can be observed after patching, and in CVE-2018-16842, line 26 can be seen to be modified. The detector Reveal successfully identifies these two vulnerable functions. However, when employing five graph-based explainers to ascertain root causes, none of them highlight statement 13 in CVE-2017-12898 or statement 26 in CVE-2018-16842 in connection to the vulnerability.

the fix indicates that the modification addresses bounds checking for the NFSv3 WRITE procedure. It checks whether the byte count is present in the captured data and whether the length of the opaque data being written is within the captured data, furthest forward in the packet, not the item before it. Following the patch, the statement in line 13, "ND_TCHECK2(dp[0], 0);", was replaced with "ND_TCHECK(dp[0]);"

Furthermore, Figure 1(b) exhibits the function "voutf" within the curl project, which is associated with CVE-2018-16842. The commit log for this fix clarifies that it corrects an issue involving improper arithmetic when outputting warnings to stderr. Specifically, we can observe that in line 26, "len -= cut"has been replaced with "len -= cut + 1;".

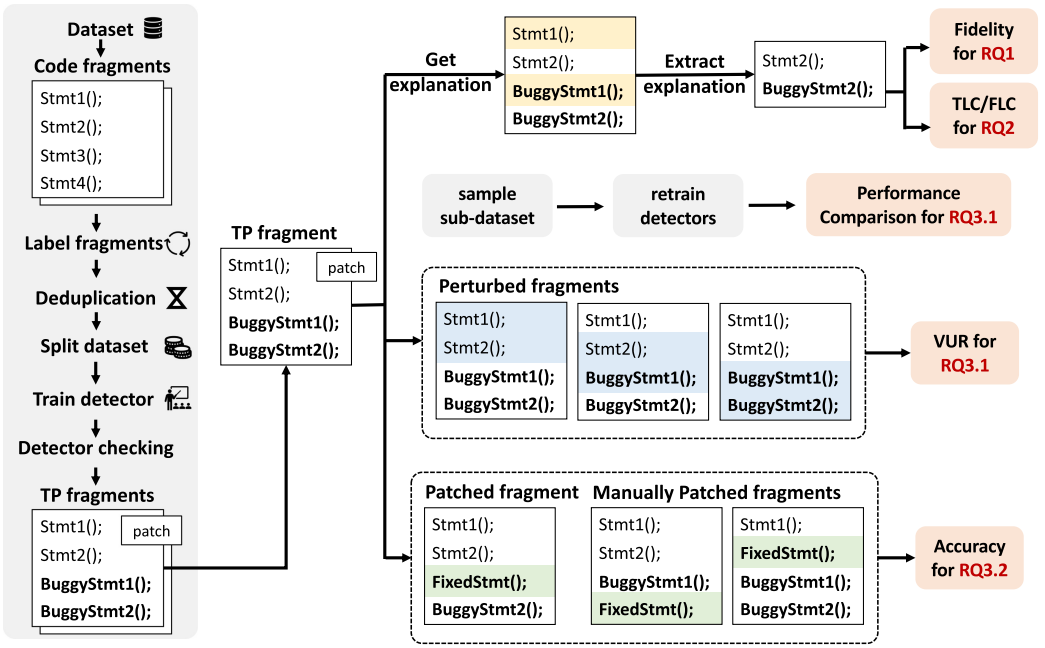


Fig. 2. Evaluation framework.

While a learning-based detector, Reveal [9], successfully identifies these two vulnerable functions, our investigation reveals that when employing five distinct explanation methods to pinpoint the root cause of the vulnerability, none of them can identify line 13 in "parserep" function or line 26 in "voutf" function as being related to the vulnerabilities. Furthermore, these five explanation methods yield diverse results, underscoring the substantial variability in their explanations.

This observation may raise questions for users, leading them to question the reliability of explanation outputs produced by explanation approaches as well as DL-based detectors. Given the limited availability of systematic studies on explanation approaches in the context of vulnerability detection, our goal is to conduct this evaluation study to address this gap in the existing literature.

4 STUDY DESIGN

In this section, we first present our evaluation framework concerning the DL-based detectors and the explanation approaches. Subsequently, we elaborate on the evaluation metrics employed for the explanation approaches in Section 4.2. Lastly, we introduce our pipeline for dataset collection and preprocessing in Section 4.3.

4.1 Evaluation Framework

Figure 2 illustrates our evaluation framework for the DL-based detectors and explanation approaches.

4.1.1 Research Questions (RQ). We address the following **research questions** (**RQs**) in this study:

***RQ1**: Can the explanation approaches accurately describe the behavior of the models?*

To answer this question, we evaluate the explainers by fidelity. A higher fidelity score indicates better performance of the explainers in descriptive accuracy [47].

***RQ2**: Can the explanation approaches provide vulnerability-relevant explanations?*

To answer this question, we assess the explainers with TLC and FLC scores. A higher TLC/FLC score reveals the better performance of the explainers in locating vulnerability-triggering statements (VTS)/vulnerability-fixing statements (VFS).

***RQ3**: Reliability of DL-based detectors?*

We further divide this RQ into two distinct sub-questions, including can explanation approaches perform better on retrained models (RQ3.1)? And can those detectors learn vulnerability-triggering mechanisms (RQ3.2)?

4.1.2 Framework Design. In this study, our objective is to explore the vulnerability localization effectiveness of explanation approaches combined with DL-based detectors, as illustrated by Figure 2. We first extract code fragments from collected datasets and label them, perform deduplication and split them into train-set, test-set, and eval-set for further analysis. Then we follow previous work [11] to employ six widely-used metrics accuracy (ACC), **false positive rate** (**FPR**), **false negative rate** (**FNR**), recall rate (R), precision (P), and F1 score (F1) to evaluate the performance of the DL-based detectors in identifying vulnerable code fragments. Through initial evaluation, we identify and select the **true positive** (**TP**) samples that the detectors successfully detect from test-set.

To evaluate the explanation approaches, we first need to assess whether these explainers can identify important features for the detectors from the samples (RQ1). Subsequently, we evaluate whether the important features selected by the explanation methods correspond to vulnerability-related statements (RQ2). For this purpose, we conduct extensive experiments on TP samples from SARD [48] and Fan datasets [49], obtaining fidelity scores and TLC/FLC scores corresponding to the explanation methods and perform further analysis. We evaluate the explanation methods only on TP samples for two main reasons. First, non-vulnerable code segments are prevalent, rendering vulnerability localization on benign samples not necessary. Second, since the explanation methods operate on top of the detectors' predictions, vulnerability localization becomes irrelevant if the detector fails to recognize a particular vulnerable code snippet.

To explore the potential acquisition of features unrelated to vulnerabilities by DL-based detectors and its effect on the reliability of explanation results, we undertake two experiments (RQ3). In the initial segment (RQ 3.1), we sample a balanced sub-dataset from the original full dataset where distinctions between vulnerable and normal samples are strictly constrained to the textual aspects of the VFS. Retraining a model on this subset while ensuring similar detection accuracy between the retrained and previous models enables us to compare the performance of the explainers. This approach aids in providing a deeper understanding of both the detectors and the explainers. Additionally, we examine the susceptibility of the detectors' predictions to be altered by modifications unrelated to vulnerabilities by analyzing the ratio of vulnerability-unrelated perturbations (VUR) among all label-flipping perturbations. In the subsequent segment (RQ 3.2), we assess whether the detector comprehends vulnerability-triggering mechanisms by analyzing the disparities in detection accuracy between the original patched samples in the dataset and the manually repaired versions of those TP samples.

4.2 Quantitative Metrics For Explanation Approaches

Evaluating explanation methods is an ongoing question in research. To comprehensively evaluate these methods, we leverage insights from reference [47] and design metrics from two perspectives: **Descriptive Accuracy**, and **Relevancy**. Descriptive Accuracy assesses if explanation methods accurately describe the behavior of models, while Relevancy determines if they provide valuable information for specific tasks. We approximate descriptive accuracy using fidelity [50] and relevancy using LC.

4.2.1 Fidelity. Fidelity[50] measures how well interpretation methods capture important features identified by the target models. Higher fidelity indicates a better reflection of the models' behavior. For sequence-based detectors, we mutate a set of k important tokens identified by explanation methods to obtain modified instances. We then calculate the prediction scores of these modified instances in the target model and track the change in the average prediction score.

$$fidelity(g, f) = \frac{1}{T}\sum_{i}^{T} f(x_i) - f\left(x_i^*\right). \tag{1}$$

In the above equation, T represents the dataset length, $g(.)$ is the explanation method, and $f(.)$ is the target vulnerability detection model. $f(x_i)$ represents the probability of sample x_i being classified as vulnerable by model $f(.)$. x_i^* denotes modified version of x_i.

For graph-based detectors, the explanation results consist of a set of k important nodes selected by explainers. We evaluate the explainers using the fidelity+ and fidelity- metrics, following the approach by Yuan et al. [45].

$$\begin{aligned} fidelity+(g, f) &= \frac{1}{T}\sum_{i}^{T} f(g_i) - f\left(g_i^{1-m_i}\right) \\ fidelity-(g, f) &= \frac{1}{T}\sum_{i}^{T} 1 - \left(f(g_i) - f\left(g_i^{m_i}\right)\right) \end{aligned}, \tag{2}$$

$g_i = G(X, E)$ represents the input graph data, where X is the embedding matrix for nodes and E is the edge set. The explanation is a discrete importance map m_i, with elements assigned 1 for critical nodes and 0 otherwise. The sum of m_i indicates the maximum number of nodes in the manually set explanation subgraph. We modify the graph data by masking feature vectors of important nodes in $g_i^{1-m_i} = G(X \cdot (1 - m_i), E)$ and retaining only the important features in $g_i^{m_i} = G(X \cdot m_i, E)$.

4.2.2 LC. The primary objective of employing the explanation approach in this context is to identify the decision basis for individual predictions and locate vulnerabilities at the statement level. If an explainer only generates a few carefully selected features that are unrelated to the vulnerability, they are of no value to explainer users. Therefore, LC is utilized to assess the correlation between the explanation results and vulnerabilities. Formally, let s^e represent the set of important tokens or nodes selected by explainers and s^v denote the set of nodes or tokens present in labeled vulnerability statements. We calculate LC in the following equation:

$$LC = \frac{|s^e \cap s^v|}{|s^v|}. \tag{3}$$

In this study, we examine LC from two perspectives. To assess the ability of explainers to identify VTS, we employ TLC. In this context, s^v represents the set of these triggering statements. Furthermore, we utilize FLC to evaluate the capability of explainers in locating VFS. Here, s^v represents the statements in the vulnerable code that have been modified in the fixed version. Based on our observations, we note that VTS are present in both vulnerable and safe codes, whereas VFS differ between the two sample types.

4.3 Dataset Construction

4.3.1 Data Source. The dataset used in this study must include annotations for lines of vulnerable code. However, certain real-world datasets like Devign and Reveal only label vulnerable functions [10], which renders them unsuitable for our specific investigation. In the case of the D2A dataset [51], it labels vulnerable traces in a program using the Infer tool [3]. Although differential

analysis on commit history is employed to reduce FPRs, many cases still result in false positives. Croft et al. [52] also highlight the presence of over two-thirds of inaccurately labeled samples in D2A, primarily due to false positives in Infer warnings and mislabeled commits. Consequently, we have chosen not to include the D2A dataset in our study.

Therefore, we constructed our dataset by combining two sources: (1) the **Software Assurance Reference Dataset** (**SARD**) [48], a widely utilized vulnerability database, and (2) the Fan dataset [49], which encompasses CVEs from 2002 to 2019 and provides 21 features for each vulnerability.

In the SARD dataset, each program or test case can be associated with one or more **Common Weakness Enumeration** (**CWE**) IDs, as a program can exhibit multiple vulnerability types. For our study, we follow Nie et al. [53] to primarily focus on six of the top 30 most critical C/C++ software weaknesses in 2021, specifically CWE20, CWE119, CWE125, CWE190, CWE400, and CWE787. To acquire the relevant programs pertaining to these vulnerabilities, we utilize the same crawler employed in DeepWuKong [11], which enables us to retrieve all the available programs associated with the aforementioned vulnerabilities.

To assess the performance of detectors and explainers on complex real-world open-source projects, we conducted additional studies using the Fan dataset. This dataset consists of over 10,000 vulnerable functions along with corresponding fixed code. It is worth noting that Croft et al. [52] have also identified inaccuracies in the labeling of the Fan dataset, primarily attributed to difficulties in accurately tracing the vulnerability-related statements in fixing commits. These inaccuracies could potentially impact the experimental results and should be taken into consideration.

4.3.2 Preprocessing of the Dataset. Once the datasets have been downloaded, the initial step involves extracting code fragments at both the function level and slice level from these programs and subsequently labeling them. We follow previous works [8–13, 15] for this step with tools Joern [54] and SVF [7].

Labeling SARD. Regarding the labeling of the SARD dataset, the VTS have already been labeled using XML files. After extracting all the code fragments from the original programs, we label each fragment as vulnerable if it contains lines that have been marked as vulnerable in the corresponding XML file, or vice versa. Simultaneously, the vulnerable lines in the vulnerable samples are labeled as **triggering lines**. However, the VFS in the vulnerable samples of SARD are not labeled. Figure 3 provides an example of a vulnerable code snippet from the SARD dataset, along with its fixed version and labels. It is noteworthy that most vulnerabilities in SARD have a fixed repair mode. Taking this into account, we have implemented an automated labeling mechanism to identify and label the fixing line in the vulnerability code fragments.

Labeling Fan. The Fan dataset is constructed by mining code change information from committed version patches to identify modified lines of code. It includes two types of modifications: added lines and deleted lines. We label the deleted lines in vulnerable functions and the lines in vulnerable functions with data or control dependencies by following the approach used in IVDetect [10]. The added lines in the patched version are considered fixed lines corresponding to the vulnerable lines. Notably, triggering lines are not labeled in Fan.

Handling duplicate/conflict samples. Duplicates in the datasets can lead to overfitting and inflated performance metrics. To address this, we follow previous work [11] to employ a method based on MD5 value comparison to remove duplicate samples. Additionally, there may be samples labeled as both vulnerable and non-vulnerable due to mislabeling in the SARD and Fan datasets. Figure 4 provides an example from the SARD dataset, where a sample is labeled as both vulnerable and non-vulnerable. These samples are typically referred to as benign in a vulnerable context within the "main" functions of SARD when labeled as vulnerable, and in a non-vulnerable context when labeled as non-vulnerable. A similar situation occurs in the Fan dataset, where some code

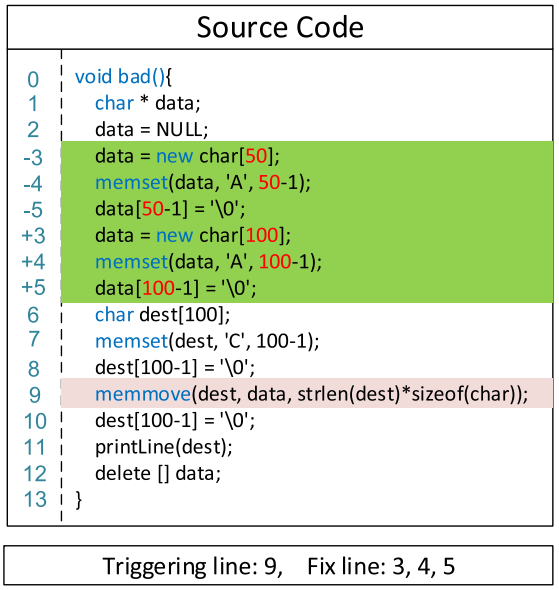


Fig. 3. Illustration of a vulnerable code fragment and its corresponding fixed version with VTS and VFS labelled.

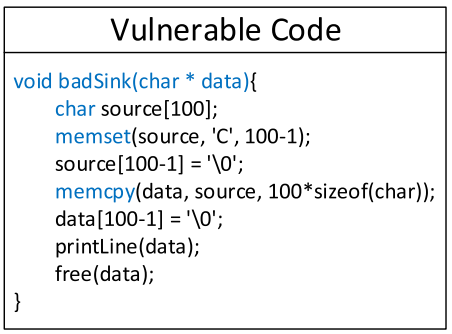


Fig. 4. An example of a controversial code fragment.

change information is extracted from consecutive patches. Consequently, a fixed code fragment may be modified again in the subsequent patch, resulting in certain functions being labeled as both vulnerable and non-vulnerable. In light of this observation, we label these conflicting samples as vulnerable. In the last, we split each dataset into train-set, eval-set, and test-set with a ratio of 80%-10%-10%.

5 EXPERIMENTS

5.1 Experimental Setup

The DL-based detectors are implemented following previous works [8–11]. We utilize utilize PyTorch and Torch_geometric for implementation of graph-based detectors, while employ Keras for sequence-based ones. The explanation approaches are implemented following Yuan et al. [45]. We first overview the detection performance of the DL-based detectors in Section 5.2. Then, in following parts, we present and analyze the experimental results for each RQ.

To facilitate the evaluation, we first derive a baseline by randomly selecting a certain number of nodes or tokens as explanation results for each sample and compute the corresponding fidelity and LC. Intuitively, the baseline gives an estimation of the fidelity and LC when a detector learns "nothing" relevant to the vulnerability. To reduce the influence of randomness, we repeat this process five times for each sample and calculate the average as the final result. We denote this method as ***Random***.

For the explanation outputs, we present the top-k results, where k can take values of 1, 3, 5, and 7. Top-k results refer to the k nodes or tokens with the highest weights for graph-based and sequence-based methods, respectively. Due to variations in the number of vulnerable lines across different datasets and the presence of multiple tokens in a labeled vulnerable line in the sequence-based approaches, the LC may not reach 100% even if all top-k results match the vulnerable lines' nodes or tokens. Considering this, we propose a benchmark value of TLC/FLC referred to as ***Expectation*** to complement our evaluation. It is calculated by assuming all the top-k results are VTS/VFS, which can be intuitively interpreted as the upper bound of TLC/FLC.

For simplicity, we abbreviate DeepWuKong, Reveal, IVDetect, Devign, SySeVR, VulDeePecker, and TokenLSTM as DWK, Rev, IVD, Dev, SySe, VDP, and TL in the figures and tables.

5.2 Vulnerability Detector Analysis

In this study, we adopt the provided settings used in the literature or re-implement them for optimal F1 scores. We stop the training process if the F1 score shows no improvements for 10 iterations. The detection results of the DL-base detectors are shown in Figure 5.

We observe that most detectors achieve high performance for the SARD dataset with F-measures exceeding 90%. VulDeePecker shows relatively lower performance due to its limited consideration of data-dependence relations and overlooking conditional judgment statements that encompass vulnerable statements. We further observe that the results do not have a significant difference among six sub-datasets from SARD in the following experiments. Therefore, we comprehensively take all sub-datasets into consideration, and present the average results over the six datasets for simplicity in the subsequent evaluations (RQ1–RQ3).

However, the performance of all detectors on the Fan dataset is significantly lower compared to that on SARD. This can be attributed to the greater complexity of code samples in real-world datasets. Nevertheless, it motivates us to investigate the features learned by these detectors. We initially hypothesized that the detectors may have learned specific text patterns unrelated to vulnerabilities that commonly appear in vulnerable code snippets.

5.3 RQ1: Can the Explanation Approaches Accurately Describe the Behavior of the Models?

In this section, our objective is to assess the significance of the explanation outcomes on DL models. To answer RQ1 we initially carry out individual experiments on graph-based and sequence-based detectors separately.

5.3.1 Fidelity Among Graph-Based Explainers. Figure 6 shows the mean experiment results of the graph-based explanation models on four detectors for both SARD and Fan datasets. We observe that:

The fidelity scores vary with calculation methods. We can observe that all explanation approaches, including Random, exhibit promising fidelity- performance. However, when interpreting the performance of the four graph-based detectors on Fan datasets, the Random shows high fidelity- but low fidelity+ scores. This indicates that random input perturbations minimally affect the model's prediction score, leading to artificially inflated fidelity- scores. Consequently,

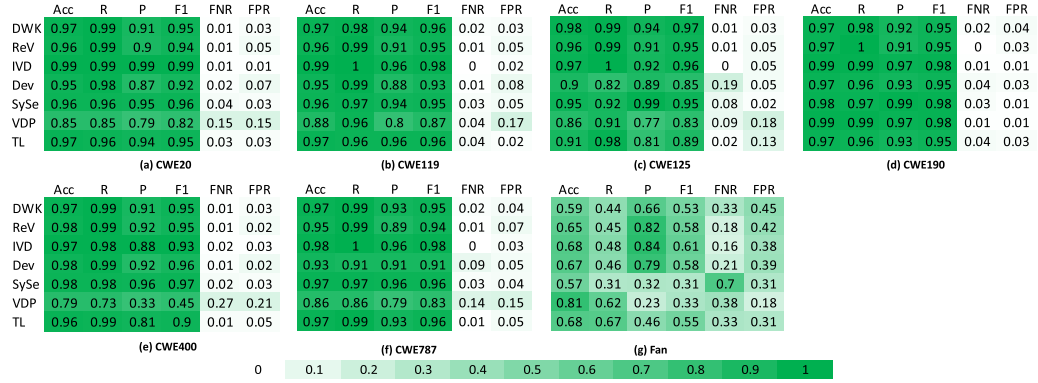


Fig. 5. Detection performance of seven DL-based detectors in different datasets.

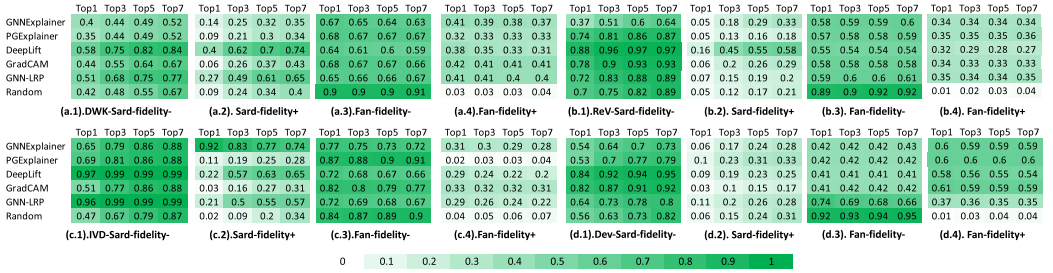


Fig. 6. Mean fidelity scores of graph-based explainers on four detectors over six vulnerability types in SARD and performance on Fan datasets.

fidelity- may not be an adequate evaluation metric. Hence in later parts, we mainly focus on fidelity+ results in the discussion due to its higher fluctuations observed.

Fidelity scores of the same explanation method vary significantly across different datasets for a given detector. We can observe that the fidelity+ scores differ between Fan and SARD datasets. This disparity is exemplified in Figure 6 (c.2) and (c.4), which present the results of all five approaches applied to explain IVDetect. Typically, explainers achieve higher fidelity scores by masking more nodes on SARD. However, this trend differs in the Fan dataset which can be illustrated in Figure 6 (b.2) and (b.4) where DeepLift is employed to explain Reveal.

The explainers exhibit varying fidelity scores when interpreting different detectors on identical datasets. For instance, as shown in Figure 6 (b.2) and (d.2), on SARD, The fidelity+ scores of GradCAM on Devign are lower compared to those on Reveal. Similarly, GNNExplainer achieves the highest fidelity+ scores for IVDetect but lower for Devign as evidenced in Figure 6 (c.2) and (d.2). This pattern can be observed in other cases as well.

5.3.2 Fidelity Among Sequence-Based Explainers. Figure 7 illustrates the fidelity results of five sequence-based explainers for detectors TokenLSTM, SySeVR, and VulDeePecker. Our results bring the following observations:

The fidelity scores of the explainers continue to exhibit variability across different detectors and datasets. Leading to differing comparisons between them. On the Sard dataset, Grad-Input achieves the highest average fidelity scores on TokenLSTM, and Lime attains the highest scores on VulDeePecker and SySeVR. While DeepLift has the lowest performance for TokenLSTM, SySeVR and VulDeePecker as shown in Figure 7 (a.1),(b.1),(c.1). On the Fan dataset, Lime emerges

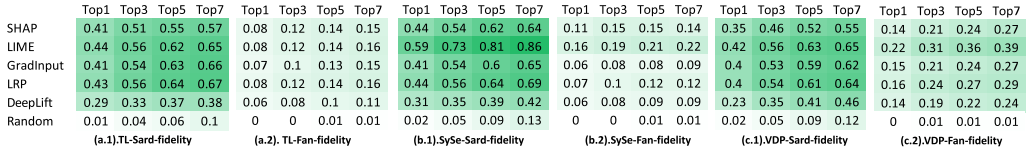


Fig. 7. Mean fidelity scores of sequence-based explainers on three detectors over six vulnerability types in SARD, and performance on Fan dataset.

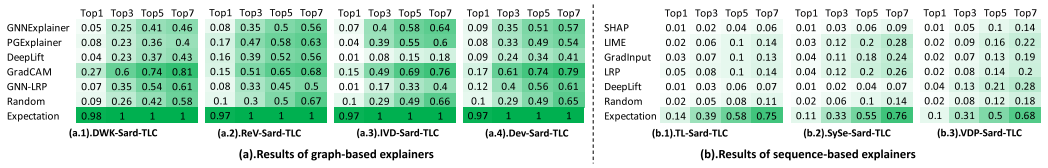


Fig. 8. Mean TLC of graph-based and sequence-based explainers on SARD datasets.

as the superior explanation approach among the five with fidelity scores as illustrated in Figure 7 (a.2),(b.2),(c.2).

The explainers generally demonstrate comparatively higher fidelity scores than Random. We can observe the fidelity scores of those explanation approaches in both SARD and Fan are significantly higher than that of Random. For instance, when explaining SySeVR on SARD, the performance of Random is much lower than the results of the worst-performing explainer DeepLift, as shown in Figure 7 (b.1).

ANSWER: The same explanation method exhibits varying performance across different detectors or datasets. It can achieve significantly better results than Random in some cases, while the opposite outcome can also be observed. As a result, determining the level of description accuracy attained by the explainer becomes challenging.

5.4 RQ2: Can the Explanation Approaches Provide Vulnerability-Relevant Explanations?

In this section, our objective is to assess the relevance of the explanation outcomes to vulnerabilities, focusing primarily on two key statements: **VTS** and **VFS**. To answer RQ2, we conducted experiments using two kinds of LC: TLC and FLC. Graph-based explainers aim at achieving the average recall rate of vulnerability-related statements, while sequence-based explainers focus on the average recall rate of tokens covered by vulnerability-related statements.

5.4.1 TLC Among Graph-Based Explainers. Figure 8 (a) presents the average TLC results for the five graph-based approaches. It allows us to observe the following patterns:

None of these graph-based explainers attain satisfactory TLC scores. We can observe that nearly all methods exhibit a significant deviation from Expectation. While the best results of GradCAM is only about 80% when explaining DeepWuKong as shown in Figure 8 (a.1), it also produces worse results on Reveal as illustrated in Figure 8 (a.2). As for the other four explainers, their performance is worse than Random in most cases.

Analyzing TLC results alongside fidelity scores reveals a weak correlation between the two. For instance, DeepLift achieves the highest fidelity+ scores when explaining DeepWuKong and Reveal as shown in Figure 6 (a.2),(b.2). Yet its TLC is lower than Random for these detectors. Notably, when explaining IVDetect, DeepLift's TLC scores are significantly lower than Random as

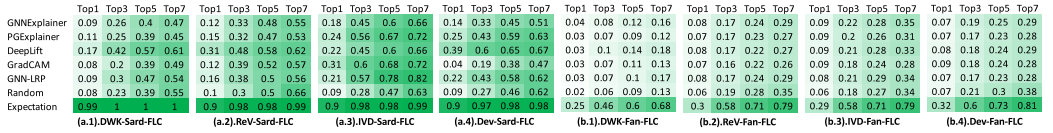


Fig. 9. Performance of graph-based explainers in terms of FLC.

illustrated in Figure 8 (a.3). On the other hand, GradCAM performs best in terms of TLC among the five approaches. Still, its fidelity+ scores are notably lower than DeepLift's when explaining the four detectors as depicted in Figure 6 (a.2),(b.2),(c.2),(d.2).

5.4.2 FLC Among Graph-Based Explainers. Figure 9 displays the experimental results of FLC for the five graph-based explainers on both the SARD and Fan datasets. The following observations can be made:

The FLC results of graph-based explainers on both datasets fall below expectations. On the Fan dataset, the FLC scores of the five explanation approaches on Reveal and Devign are notably lower than Random as shown in Figure 9 (b.2),(b.4), not to mention the comparison with Expectation. When explaining DeepWuKong and IVDetect, DeepLift and GNNExplainer perform the best over other methods, respectively. However, their performance still falls considerably short, being even worse than half of Expectation as shown in Figure 9 (b.1),(b.3). On the SARD dataset, DeepLift attains the best FLC scores when explaining DeepWuKong, Reveal, and Devign. Nevertheless, all its top-7 FLC are below 70%. As for GNN-LRP, although it achieves the highest top-7 FLC of 82% when explaining IVDetect, as shown in Figure 9 (a.3), there is a significant decrease in performance when explaining the other three detectors.

The results of the explanation methods on FLC show little correlation to TLC and fidelity. On both SARD and Fan dataset, the fidelity+ scores and FLC scores do not align. Despite GNNExplainer achieving the highest fidelity+ scores when explaining IVDetect shown in Figure 6 (c.2), its FLC results are worse than PGExplainer, GradCAM, and GNN-LRP in this case depicted in Figure 9 (a.3). On the Fan dataset, when explaining IVDetect, GNNExplainer performs 1%-4% better than Random as depicted in Figure 9 (b.3). However, there is a significant difference in fidelity+ scores between Random and the other methods as shown in Figure 6 (c.4), indicating a lack of correlation between FLC and fidelity+. Similarly, the FLC scores do not correspond to the TLC scores. While GradCAM achieves the highest TLC scores on DeepWuKong as illustrated in Figure 8 (a.1), its corresponding FLC scores are significantly lower than those of DeepLift and GNN-LRP as shown in Figure 9 (a.1).

5.4.3 TLC Among Sequence-Based Explainers. Figure 8 (b) presents the TLC scores of sequence-based explainers, indicating:

Sequence-based explainers obtain unsatisfactory TLC scores. Similar to the situation in graph-based explainers, the TLC scores of the sequence-based explainer significantly deviate from the Expectation. When explaining SySeVR, Lime produces the highest TLC scores, which are nearly twice than Random as shown in Figure 8 (b.2), however, the scores are even lower than half of the Expectation. The same observation holds true for the other two detectors.

The fidelity scores of sequence-based explainers exhibit a limited correlation with TLC scores. Analyzing the results in terms of fidelity scores, we observe that higher fidelity scores do not necessarily correspond to higher TLC. Taking Lime, for example, it attains the highest fidelity scores among five sequence-based explainers as shown in Figure 7 (c.1). However, in regard to TLC scores, it is inferior to DeepLift on VulDeePecker as depicted in Figure 8 (b.3), while DeepLift produces significantly lower fidelity scores than Lime.

	Top1	Top3	Top5	Top7
SHAP	0.1	0.16	0.19	0.23
LIME	0.06	0.1	0.14	0.17
GradInput	0.08	0.13	0.15	0.17
LRP	0.07	0.11	0.14	0.17
DeepLift	0.08	0.14	0.19	0.22
Random	0.02	0.05	0.08	0.11
Expectation	0.21	0.58	0.77	0.85

(a.1).TL-Sard-FLC

	Top1	Top3	Top5	Top7
SHAP	0.07	0.14	0.19	0.24
LIME	0.07	0.13	0.17	0.21
GradInput	0.05	0.1	0.15	0.19
LRP	0.05	0.1	0.14	0.18
DeepLift	0.06	0.14	0.2	0.25
Random	0.02	0.06	0.11	0.15
Expectation	0.14	0.41	0.68	0.86

(a.2).SySe-Sard-FLC

	Top1	Top3	Top5	Top7
SHAP	0.08	0.17	0.23	0.28
LIME	0.07	0.14	0.18	0.23
GradInput	0.07	0.15	0.18	0.23
LRP	0.07	0.13	0.19	0.23
DeepLift	0.02	0.05	0.07	0.11
Random	0.03	0.07	0.13	0.18
Expectation	0.13	0.4	0.66	0.83

(a.3).VDP-Sard-FLC

	Top1	Top3	Top5	Top7
SHAP	0.02	0.05	0.07	0.09
LIME	0.01	0.03	0.05	0.07
GradInput	0.01	0.02	0.03	0.04
LRP	0.01	0.02	0.03	0.04
DeepLift	0.01	0.04	0.05	0.07
Random	0.01	0.02	0.04	0.05
Expectation	0.05	0.15	0.23	0.3

(b.1).TL-FAN-FLC

	Top1	Top3	Top5	Top7
SHAP	0.01	0.02	0.04	0.05
LIME	0	0.01	0.02	0.03
GradInput	0	0.01	0.02	0.03
LRP	0	0.01	0.02	0.02
DeepLift	0.01	0.02	0.03	0.04
Random	0	0.01	0.02	0.03
Expectation	0.07	0.15	0.21	0.28

(b.2). SySe-FAN-FLC

	Top1	Top3	Top5	Top7
SHAP	0.01	0.02	0.03	0.05
LIME	0	0.01	0.02	0.03
GradInput	0	0.02	0.02	0.03
LRP	0.01	0.01	0.02	0.03
DeepLift	0	0.01	0.02	0.02
Random	0	0.01	0.02	0.03
Expectation	0.05	0.14	0.19	0.23

(b.3).VDP-FAN-FLC

Fig. 10. Performance of sequence-based explainers in terms of FLC.

5.4.4 FLC Among Sequence-Based Explainers. Figure 10 illustrates the FLC results of sequence-based explainers, it reveals that:

The FLC scores achieved by sequence-based explainers fall short of expectations. On the Fan dataset, when explaining TokenLSTM, Shap attains the best FLC score, while it only reaches 30% of Expectation as shown in Figure 10 (b.1). On the SARD dataset, Shap performs the best in FLC among the five explainers, however, the observed outcome diverges significantly from Expectation as depicted own in Figure 10 (a.1),(a.2),(a.3).

There is minimal correspondence between the fidelity scores and FLC. One example is that Shap continues to outperform other explainers in terms of FLC on the SARD dataset. However, it does not achieve the highest fidelity scores across all detectors as shown in Figure 7 (a.1),(b.1),(c.1).

Considering the aforementioned observations, it is uncertain whether explanation methods can reliably provide valuable insights. We hypothesize that DL-based detectors aim at learning the distinctions between vulnerable and normal samples, attempting to discern whether the content represents a vulnerability or normal code fragments. Given that VTSs exist in both vulnerable and non-vulnerable programs, DL-based detectors might not prioritize the importance of VTS in determining vulnerabilities. Hence, theoretically, explanation methods should achieve high FLC. As they focus on the differences between vulnerable and normal samples. However, experimental outcomes do not consistently align with this expectation. Furthermore, there appears to be no direct correlation observed among fidelity, TLC, and FLC values. As suggested by T Ganz [55], this discrepancy might stem from DL-based detectors being trained not to recognize vulnerabilities but artifacts encompassing not only VFS. These findings underscore the need for further analysis of DL-based detectors and explanation approaches, which will be discussed in Section 5.5.

ANSWER: The LC scores of these explanations indicate their failure to provide sufficient vulnerability context in some cases, rendering them unsuitable for working with DL-based detectors in identifying vulnerability-related statements.

5.5 RQ3: Reliability of DL-Based Detectors

In this section, we investigate whether DL-based detectors have learned features unrelated to vulnerabilities, leading to unreliable explanation results. This research comprises two parts. In the first segment (RQ 3.1), we initially sample a sub-dataset from the original full dataset, ensuring an equal quantity of vulnerable and non-vulnerable samples while controlling the differences between vulnerable and normal samples primarily within VFS. This sub-dataset is partitioned into train, evaluation, and test sets. We retrain a model, comparing the detection performance of the retrained model with the previous model on the sub-test dataset. When the performance difference between the retrained and previous models is minimal, we compare the performance differences of explanation methods between the retrained and previous models. Additionally, we introduce two metrics to measure the sensitivity of the retrained and previous models to VTS and VFS, as well as their susceptibility to disturbances irrelevant to vulnerabilities. In the second part (RQ 3.2), we aim at evaluating the detectors' capacity to identify previously unseen vulnerability-fixing

Table 3. Number of Vulnerable Functions in Each Testset After Filtering

CWE-20	CWE-119	CWE-125	CWE-190	CWE-400	CWE-787
804	875	172	91	41	664

patterns. This assessment intends to discern whether DL-based detectors genuinely comprehend vulnerabilities or merely recognize artifacts.

5.5.1 RQ3.1: Comparsion Between Previous and Retrained Model.

Experimental Setting. In this part, we aim at investigating the impact of DL models themselves on the explanation methods' ability to pinpoint vulnerable statements. We approach this by first sampling a sub-dataset from the full dataset, ensuring pairs of vulnerable and normal samples while emphasizing differences primarily within the vulnerability-fixing lines. Subsequently, we divide this sub-dataset into three segmentstrain, eval, and testfollowing an 8:1:1 ratio. Based on this sub-dataset, we retrain a model, ensuring similar detection performance between the retrained and previous models. Under this condition, we experiment with explanation methods. To measure their performance, we employ three metrics: fidelity, TLC, and FLC. For graph-based explainer, we utilize fidelity+ as a supplemental fidelity metric. Theoretically, the retrained model, due to the more pronounced differences between vulnerable and normal samples in the training set, should display increased sensitivity to VFS after retraining. Consequently, this heightened sensitivity should enhance the explanation methods' ability to pinpoint parts related to vulnerabilities.

Dataset and detector consideration. When sampling the dataset, we chose to sample from the SARD portion and not from the Fan dataset. This decision was influenced by concerns regarding the potential presence of vulnerability-unrelated code changes in patch commits, as discussed in [52]. Such changes could result in inaccurate labeling of vulnerabilities within the dataset. Simultaneously, we solely employ function-level detectors for experimentation and refrain from using slice-level detectors. This decision is based on the fact that DeepWuKong, SySeVR, and VulDeePecker detectors conduct slicing and symbolization operation. This process potentially exacerbates the differences unrelated to vulnerabilities between vulnerable and normal samples. For example, during symbolization, variable names such as "data" or "databuffer" in the original samples may be symbolized as "VAR1" and "VAR2". However, in different samples or between a vulnerability sample and its corresponding normal sample, these variables may be symbolized as different values, such as "VAR1" in the vulnerability sample and "VAR2" in the normal sample. Subsequent experiments (Section 5.5.2) reveal that these textual disparities interfere to some extent with DL-based detectors.

Balanced sub-dataset curation. For the samples within the full SARD dataset, we collect each vulnerable function and its corresponding patched version based on the test case ID. Subsequently, we extract samples with the same number of lines in both the vulnerable and normal functions, forming a new sub-dataset. This process entails some lines within the vulnerable functions being replaced by others in the patched version. This method is employed considering our observation: within SARD, vulnerable functions and their corresponding patched functions, with an identical number of lines, generally exhibit similar graph structures or possess token sequences with comparable lengths. This practice helps to minimize differences related to vulnerability-unrelated aspects between the vulnerable and normal samples as much as possible.

Dataset and detector selection. After sampling, the remaining vulnerable sample sizes in the sub-test set are shown in Table 3, indicating a considerable reduction in the dataset sizes for each type of vulnerability. The most notable reductions are observed in CWE-190 and CWE-400. On the sub-dataset, we first retrain the models for Reveal, IVDetect, Devign, and TokenLSTM.

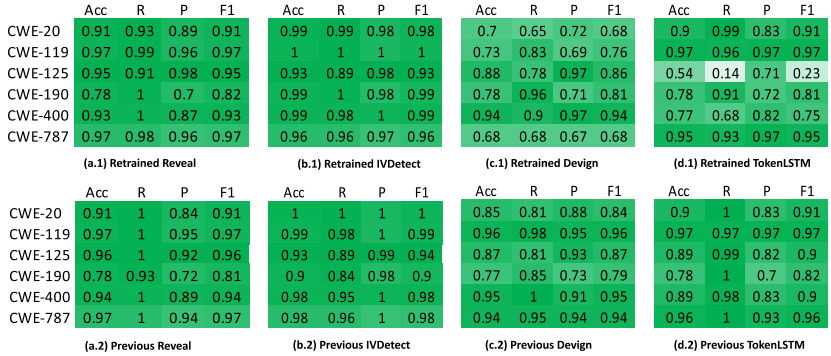


Fig. 11. Detection performance of four function-level detectors with retrained and previous models on sub-test-dataset.

The comparison of the performance on the sub-test dataset between the previous and retrained models for each detector is depicted in Figure 11. We can observe that Reveal and IVDetect exhibit minor performance differences between the previous and retrained models across the six types of sub-test datasets. However, Devign shows performance declines in CWE-20, CWE-119, and CWE-787, while TokenLSTM specifically demonstrates reduced performance in CWE-125, CWE-190, and CWE-400. We speculate that the performance declines may be related to the quantity and quality of the training samples. Considering the requirement for similar performance between the previous and retrained models, the number of test samples, and the need to involve both graph-based and sequence-based methods, we opt to continue the experiments on portions of CWE-20 and CWE-119 samples for Reveal, IVDetect, and TokenLSTM.

VUR metric. To explore the susceptibility of DL-based detectors to perturbations unrelated to vulnerabilities, we introduce a novel metric termed VUR (Vulnerability Unrelated Ratio). VUR quantifies the model's susceptibility to perturbations unrelated to vulnerabilities and is determined by the following equation:

$$VUR = \frac{\sum_{sc_i}(\hat{f}(x) - \hat{f}(x^{-sc_i})).(|sc_i \cap (VFS \cup VTS)| = 0)}{\sum_{sc_i}(\hat{f}(x) - \hat{f}(x^{-sc_i}))}. \tag{4}$$

Here, $\hat{f}(x) = (f(x) > 0.5)$ represents the predicted label of the corresponding detector f, and sc_i denotes a set of nodes or tokens associated with a perturbation. We exclusively calculate VUR on TP samples. Thus, $\hat{f}(x) = 1$. When $\hat{f}(x^{-sc_i}) = 0$, it signifies that the perturbation sc_i has successfully flipped the model's prediction. Hence, the denominator in Equation (4) denotes the number of perturbations within a given perturbation set capable of flipping the model's prediction. The term $(|sc_i \cap (VFS \cup VFS)| = 0)$ signifies whether sc_i has no intersection with VFS or VTS, which equates to 1 if there is no intersection. Therefore, the numerator in Equation (4) computes the count of perturbations within the perturbation set capable of flipping the model's prediction and having no intersection with VFS or VTS. VUR signifies the proportion of perturbations capable of flipping the model's predicted label, which have no intersection with VTS and VFS. A higher VUR value indicates a greater susceptibility of the model to be influenced by perturbations that are unrelated to vulnerabilities. Computing the VUR for all potential perturbations for each sample entails a significant computational cost. Therefore, in our computations, we randomly select 100 perturbations, each ranging in length from 1 to 3, for every sample to calculate the respective VUR. As a result, the VUR values for the retrained and previous models are not

	Top1	Top3	Top5	Top7
GNNExplainer	0.02	0.08	0.12	0.14
PGExplainer	0.03	0.09	0.11	0.11
DeepLift	0.16	0.39	0.49	0.51
GradCAM	0.08	0.18	0.2	0.21
GNN-LRP	0.11	0.2	0.27	0.28

(a.1).fidelity of RM

	Top1	Top3	Top5	Top7
GNNExplainer	0.07	0.33	0.49	0.54
PGExplainer	0.01	0.11	0.16	0.17
DeepLift	0.18	0.31	0.37	0.41
GradCAM	0.08	0.23	0.35	0.4
GNN-LRP	0.1	0.4	0.57	0.62

(b.1).TLC of RM

	Top1	Top3	Top5	Top7
GNNExplainer	0.12	0.34	0.5	0.56
PGExplainer	0.11	0.35	0.46	0.5
DeepLift	0.48	0.72	0.78	0.8
GradCAM	0.25	0.51	0.64	0.68
GNN-LRP	0.36	0.63	0.76	0.8

(c.1).FLC of RM

	Top1	Top3	Top5	Top7
GNNExplainer	0.05	0.15	0.26	0.31
PGExplainer	0.07	0.21	0.37	0.42
DeepLift	0.2	0.52	0.7	0.74
GradCAM	0.09	0.26	0.4	0.45
GNN-LRP	0.12	0.31	0.47	0.53

(a.2).fidelity of PM

	Top1	Top3	Top5	Top7
GNNExplainer	0.06	0.31	0.5	0.56
PGExplainer	0.23	0.56	0.7	0.73
DeepLift	0.16	0.5	0.71	0.75
GradCAM	0.14	0.44	0.56	0.61
GNN-LRP	0.12	0.39	0.58	0.64

(b.2).TLC of PM

	Top1	Top3	Top5	Top7
GNNExplainer	0.15	0.37	0.52	0.57
PGExplainer	0.15	0.38	0.59	0.65
DeepLift	0.41	0.63	0.72	0.73
GradCAM	0.05	0.25	0.37	0.4
GNN-LRP	0.26	0.52	0.69	0.73

(c.2).FLC of PM

	Top1	Top3	Top5	Top7
GNNExplainer	-0.03	-0.07	-0.14	-0.17
PGExplainer	-0.04	-0.12	-0.26	-0.31
DeepLift	-0.04	-0.13	-0.21	-0.23
GradCAM	-0.01	-0.08	-0.2	-0.23
GNN-LRP	-0.01	-0.11	-0.2	-0.25

(a.3).comparison of fidelity

	Top1	Top3	Top5	Top7
GNNExplainer	0.01	0.02	-0.01	-0.02
PGExplainer	-0.22	-0.45	-0.54	-0.56
DeepLift	0.02	-0.19	-0.34	-0.34
GradCAM	-0.06	-0.21	-0.21	-0.21
GNN-LRP	-0.02	0.01	-0.01	-0.02

(b.3).comparison of TLC

	Top1	Top3	Top5	Top7
GNNExplainer	-0.03	-0.03	-0.02	-0.01
PGExplainer	-0.04	-0.03	-0.13	-0.15
DeepLift	0.07	0.09	0.06	0.07
GradCAM	0.2	0.26	0.27	0.28
GNN-LRP	0.1	0.11	0.09	0.07

(c.3).comparison of FLC

Fig. 12. Explanation results for Reveal. "RM" denotes "retrained model", "PM" denotes "previous model", "comparison" denotes the performance comparison between retrained and previous models.

directly comparable. Instead, they indicate the susceptibility of different models to disturbances irrelevant to vulnerabilities. In this context, VUR serves as a valuable metric for determining the suitability of perturbation-based evaluation metrics and explanation approaches in the context of vulnerability detection. An elevated VUR score suggests that higher-fidelity explanations may not necessarily convey vulnerability-related information.

To further probe the importance of the VTS and VFS for DL-based detectors. We define a metric called *imp* which is similar to fidelity. It could be denoted as $imp_{VTS/VFS} = f(x) - f(x^{-VTS/VFS})$, where $imp_{VTS/VFS}$ indicates the importance score for VTS or VFS, x is an input code fragment, $x^{-VTS/VFS}$ is the sample after masking VTS or VFS.

Experimental Results. The explanation results for Reveal, IVDetect, and TokenLSTM are depicted in Figures 12, 13, and 14, respectively. From the results, we can observe:

Firstly, perturbation-based evaluation metrics or explanation approaches may not be suitable for vulnerability detection. It is evident that the level of fidelity does not inherently correlate with FLC and TLC. In the majority of explanation methods, despite achieving higher FLC on the retrained model compared to the previous model, their fidelity values tend to decrease. Specifically in the context of explaining IVDetect, both DeepLift and GNNLRP exhibited respectable fidelity on the previous model. However, GNNLRP displayed higher TLC and FLC compared to DeepLift. Therefore, fidelity might not be a reliable metric as disturbances irrelevant to vulnerabilities could significantly impact the model's predictions.

The VUR results for retrained and previous detectors are presented in Table 4. Notably, in both models, a significant proportion of perturbations unrelated to vulnerabilities have the capacity to invert the detectors' labels. We perform an assessment of all detectors using the previous models on the full-test dataset, and the outcomes are detailed in Table 5. Remarkably, this assessment reiterates that vulnerability-unrelated perturbations possess the capability to flip the predictions of all detectors. Consequently, explanations with high fidelity might not necessarily be correlated with vulnerabilities. Therefore, using fidelity as an evaluation metric in the context of vulnerability

	Top1	Top3	Top5	Top7
GNNExplainer	0.89	0.76	0.66	0.6
PGExplainer	0.12	0.31	0.4	0.44
DeepLift	0.38	0.87	0.89	0.89
GradCAM	0.06	0.33	0.63	0.69
GNN-LRP	0.38	0.78	0.83	0.84

(a.1).fidelity of RM

	Top1	Top3	Top5	Top7
GNNExplainer	0.02	0.39	0.61	0.69
PGExplainer	0.01	0.28	0.47	0.54
DeepLift	0	0.07	0.18	0.26
GradCAM	0.12	0.31	0.57	0.67
GNN-LRP	0	0.22	0.42	0.52

(b.1).TLC of RM

	Top1	Top3	Top5	Top7
GNNExplainer	0.18	0.46	0.62	0.68
PGExplainer	0.14	0.64	0.74	0.79
DeepLift	0.32	0.48	0.56	0.58
GradCAM	0.49	0.77	0.87	0.89
GNN-LRP	0.31	0.8	0.91	0.93

(c.1).FLC of RM

	Top1	Top3	Top5	Top7
GNNExplainer	0.9	0.77	0.64	0.57
PGExplainer	0.14	0.32	0.45	0.5
DeepLift	0.35	0.89	0.93	0.93
GradCAM	0.05	0.3	0.53	0.62
GNN-LRP	0.34	0.77	0.85	0.86

(a.2).fidelity of PM

	Top1	Top3	Top5	Top7
GNNExplainer	0.03	0.39	0.61	0.68
PGExplainer	0.01	0.29	0.49	0.56
DeepLift	0.01	0.07	0.12	0.14
GradCAM	0.11	0.31	0.61	0.71
GNN-LRP	0.01	0.19	0.41	0.51

(b.2).TLC of PM

	Top1	Top3	Top5	Top7
GNNExplainer	0.16	0.45	0.63	0.69
PGExplainer	0.14	0.53	0.66	0.72
DeepLift	0.27	0.55	0.68	0.7
GradCAM	0.4	0.74	0.83	0.85
GNN-LRP	0.27	0.79	0.89	0.9

(c.2).FLC of PM

	Top1	Top3	Top5	Top7
GNNExplainer	-0.01	-0.01	0.02	0.03
PGExplainer	-0.02	-0.01	-0.05	-0.06
DeepLift	0.03	-0.02	-0.04	-0.04
GradCAM	0.01	0.03	0.1	0.07
GNN-LRP	0.04	0.01	-0.02	-0.02

(a.3).comparison of fidelity

	Top1	Top3	Top5	Top7
GNNExplainer	-0.01	0	0	0.01
PGExplainer	0	-0.01	-0.02	-0.02
DeepLift	-0.01	0	0.06	0.12
GradCAM	0.01	0	-0.04	-0.04
GNN-LRP	-0.01	0.03	0.01	0.01

(b.3).comparison of TLC

	Top1	Top3	Top5	Top7
GNNExplainer	0.02	0.01	-0.01	-0.01
PGExplainer	0	0.11	0.08	0.07
DeepLift	0.05	-0.07	-0.12	-0.12
GradCAM	0.09	0.03	0.04	0.04
GNN-LRP	0.04	0.01	0.02	0.03

(c.3).comparison of FLC

Fig. 13. Explanation results for IVDetect.

	Top1	Top3	Top5	Top7
SHAP	0.45	0.5	0.52	0.52
LIME	0.45	0.49	0.52	0.53
GradInput	0.35	0.41	0.45	0.49
LRP	0.36	0.43	0.47	0.5
DeepLift	0.26	0.3	0.34	0.38

(a.1).fidelity of RM

	Top1	Top3	Top5	Top7
SHAP	0.02	0.03	0.05	0.06
LIME	0.01	0.04	0.07	0.1
GradInput	0.02	0.03	0.05	0.07
LRP	0.02	0.03	0.05	0.07
DeepLift	0.01	0.02	0.03	0.04

(b.1).TLC of RM

	Top1	Top3	Top5	Top7
SHAP	0.1	0.2	0.27	0.34
LIME	0.1	0.15	0.19	0.23
GradInput	0.09	0.16	0.23	0.28
LRP	0.1	0.17	0.24	0.29
DeepLift	0.07	0.13	0.17	0.21

(c.1).FLC of RM

	Top1	Top3	Top5	Top7
SHAP	0.54	0.59	0.63	0.64
LIME	0.54	0.58	0.58	0.58
GradInput	0.66	0.69	0.72	0.75
LRP	0.67	0.7	0.74	0.76
DeepLift	0.5	0.5	0.53	0.53

(a.2).fidelity of PM

	Top1	Top3	Top5	Top7
SHAP	0.01	0.03	0.04	0.06
LIME	0.02	0.05	0.08	0.12
GradInput	0.02	0.04	0.06	0.08
LRP	0.02	0.04	0.06	0.08
DeepLift	0.01	0.03	0.04	0.06

(b.2).TLC of PM

	Top1	Top3	Top5	Top7
SHAP	0.08	0.15	0.18	0.21
LIME	0.06	0.11	0.15	0.18
GradInput	0.08	0.16	0.19	0.22
LRP	0.08	0.16	0.19	0.23
DeepLift	0.07	0.14	0.19	0.23

(c.2).FLC of PM

	Top1	Top3	Top5	Top7
SHAP	-0.09	-0.09	-0.11	-0.12
LIME	-0.09	-0.09	-0.06	-0.05
GradInput	-0.31	-0.28	-0.27	-0.26
LRP	-0.31	-0.27	-0.27	-0.26
DeepLift	-0.24	-0.2	-0.19	-0.15

(a.3).comparison of fidelity

	Top1	Top3	Top5	Top7
SHAP	0.01	0	0.01	0
LIME	-0.01	-0.01	-0.01	-0.02
GradInput	0	-0.01	-0.01	-0.01
LRP	0	-0.01	-0.01	-0.01
DeepLift	0	-0.01	-0.01	-0.02

(b.3).comparison of TLC

	Top1	Top3	Top5	Top7
SHAP	0.02	0.05	0.09	0.13
LIME	0.04	0.04	0.04	0.05
GradInput	0.01	0	0.04	0.06
LRP	0.02	0.01	0.05	0.06
DeepLift	0	-0.01	-0.02	-0.02

(c.3).comparison of FLC

Fig. 14. Explanation results for TokenLSTM.

detection might not be suitable. Additionally, existing perturbation-based methods like GNNExplainer, PGExplainer, and so on, which aim at maximizing the impact on model prediction metrics, are also unsuitable for deployment in the realm of vulnerability detection.

DL-based detectors are focused on differentiating vulnerable and normal samples, a characteristic also mirrored by gradient-, decomposition-, and surrogate-based explainers. Within Reveal, all model-related explainers demonstrate an ability to achieve higher FLC

Table 4. VUR Results of Detectors. "R" Denotes Retrained Model, "P" Demotes Previous Model

R-ReV	P-ReV	R-IVD	P-IVD	R-TL	P-TL
0.47	0.52	0.48	0.26	0.36	0.37

Table 5. VUR Results of Detectors with Previous Models on Full Datasets

ReV	IVD	Dev	DWK	SySe	VDP	TL
0.78	0.90	0.57	0.71	0.66	0.43	0.78

scores while concurrently registering decreased TLC. Regarding IVDetect, while the FLC score of DeepLift remains relatively modest without notable improvements, both GradCAM and GNN-LRP consistently maintain notably high FLC scores, exhibiting a slight increase, particularly evident in the retrained model. In the case of TokenLSTM, it is notable that all deployed explainers, with the exception of DeepLift, attain higher FLC scores in the retrained models. The aforementioned observations suggest that the retrained model indeed tends to place more emphasis on the VFS during predictions compared to the previous model. Additionally, explanation methods, beyond perturbation-based approaches, to some extent, can capture this particular aspect of the model. The results depicted in Table 6 reveal that in both the retrained and previous models, the significance of VFS surpasses that of VTS. Nevertheless, when examining their explanations in both retrained and previous models, their results may still not be considered ideal. This implies that they are still prone to selecting features unrelated to vulnerabilities, which could be associated with both the deep learning models and the explanation methods themselves. However, discussing this aspect is beyond the scope of our study.

For these three detectors, the imp_{VTS} in the retrained model has decreased, aligning with our hypothesis that "VTS exist in both vulnerable and normal samples, leading the detectors to not assign high importance during prediction." However, concerning imp_{VFS}, in the case of IVDetect and TokenLSTM, the scores have improved after retraining. In contrast, Reveal shows a slight decline after retraining, deviating from our expectations. Yet, considering the earlier discussion, perturbation-based evaluation metrics might not robustly quantify the significance of VFS and VTS for the model. Moreover, given the heightened FLC scores of the model-related explainers-DeepLift, GradCAM, and GNN-LRP in Reveal, it consistently indicates the model's inclination toward assigning greater importance to VFS during predictions on the sub-test dataset. Notably, these model-related explainers effectively identify and highlight this particular aspect. However, the decrease in fidelity in the retrained model indicates that even if the model captures this fact, it might not necessarily be quantifiable through perturbation-based metrics.

ANSWER: DL-based detectors are susceptible to perturbations unrelated to vulnerabilities, resulting in the inversion of predicted labels. Hence, fidelity, a perturbation-based evaluation metric, is not suitable for assessing the performance of explanation methods in the context of vulnerability detection. Additionally, existing perturbation-based explainers are not applicable in this scenario. Furthermore, explainers beyond perturbation-based ones have the capacity to capture information related to the models' prediction process. Nonetheless, all explanation approaches are still susceptible to selecting features unrelated to vulnerabilities.

5.5.2 RQ3.2: Probing DL-Based Detectors. In our previous experiments, we observed that existing DL-based detectors are susceptible to disturbances unrelated to vulnerabilities. Moreover, they

Table 6. imp_{VTS} and imp_{VFS} for Retrained and Previous Detectors. Higher *imp* Scores Reflect the Significance of VTS/VFS in Model Decision-Making

Detector	R-ReV	P-ReV	R-IVD	P-IVD	R-TL	P-TL
imp_{VTS}	0.04	0.094	0.001	0.03	0.114	0.14
imp_{VFS}	0.154	0.179	0.297	0.214	0.59	0.527

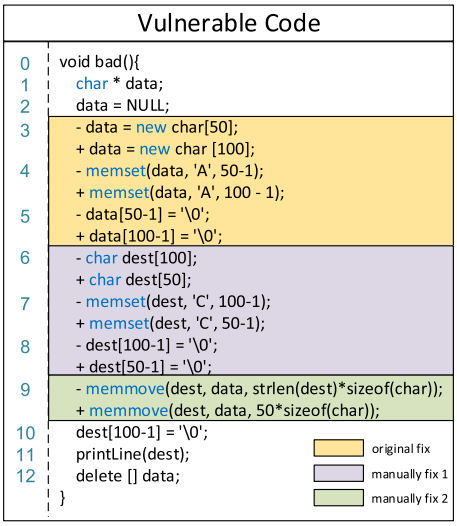


Fig. 15. Examples of the original fixed version and manually fixed version.

exhibit a clearer distinction between vulnerability and normal samples, relying predominantly on VFS for predictions, particularly if there are almost no discrepancies present in the training set apart from the VFS. However, what remains unclear is whether DL-based detectors merely discern differences between positive and negative samples to identify vulnerable code, or if they genuinely comprehend the mechanisms triggering vulnerabilities. In this part, our objective is to evaluate the detectors' capacity to recognize previously unseen vulnerability-fixing patterns. This analysis aims to provide further insights into this particular aspect.

We manually selected 100 TP samples and their corresponding fixed versions from each sub-dataset of SARD. Various methods were employed to manually correct the vulnerable samples. For example, Figure 15 showcases a piece of patched vulnerable code example, "-" denoted removed lines from vulnerable code, "+" denotes added lines in the patched version. Where lines 3–5 are originally fixed ways. Lines 6–8 and 9 are two different manually fixed ways. During the manual correction process, we ensured that all the tokens incorporated into the code were already present in the original SARD dataset to avoid the **out-of-vocabulary** (**OOV**) problem of the embedding models. Subsequently, we evaluated both corrected code versions using vulnerability detectors and compared their respective accuracies. Following experiments in Section 5.5.1, we opt not to carry out this experiment on the Fan dataset for the same reason.

The results are presented in Table 7, highlighting a significant decrease in accuracy for the detectors when evaluated on the manually corrected samples compared to the original corrected samples. The difference in results ranges from 55% to 87% across each detector. These findings suggest that the DL-based detectors are unable to recognize previously unseen fixing patterns, indicating a lack of effectiveness in capturing the underlying triggering mechanism of vulnerabilities.

Table 7. Comparsion of Accuracy Between Original Fixed Samples and Manual Fixed Samples

fix type	ReV	IVD	Dev	DWK	SySe	VDP	TL
original fix	0.74	0.92	0.89	0.97	0.98	0.86	0.95
manually fix	0.19	0.32	0.20	0.10	0.19	0.14	0.11

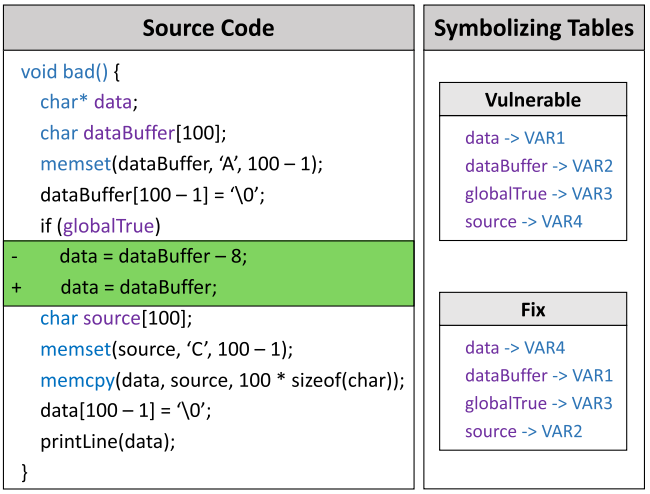


Fig. 16. Examples illustrating problems of symbolizing.

Furthermore, during this experiment, we incidentally observe a text pattern capable of inducing misclassifications for DeepWuKong, as illustrated in Figure 16. It displays a vulnerable code snippet and its corresponding fixed version. During preprocessing for DeepWuKong, variable names are transformed into symbolic names, such as "VAR" followed by an index. However, issues in the preprocessing pipeline have resulted in different symbolic names being assigned to identical variables in the vulnerable and fixed codes. Surprisingly, DeepWuKong can still accurately predict the symbolized samples. To investigate further, we manually adjusted the symbolization by changing the variable order. Remarkably, DeepWuKong misclassifies both the vulnerable and fixed code in this scenario. This observation indicates that vulnerability detectors rely more on recognizing textual patterns rather than truly comprehending underlying vulnerability patterns. Consequently, this limitation may hinder the ability of explanation approaches to identify vulnerability lines.

ANSWER: DL-based detectors primarily rely on differences learned from the training set between vulnerable and normal samples, notably emphasizing VFS to identify vulnerabilities rather than discerning the triggering mechanisms of vulnerabilities. Also, DL-based detectors may learn vulnerability-unrelated features during training process.

6 DISCUSSIONS

6.1 Limitations

First, we are not rigorously studying whether these explanation methods descriptively and accurately explain the models' output. In our study, we attempted to employ perturbation-based methods to compute the fidelity of explanation results and assess the significance of VTS and VFS in determining the model's importance. However, subsequent observations revealed a paradoxical decrease in

fidelity despite the retrained model showing higher FLC. Moreover, the model is indeed susceptible to perturbations that can alter the predicted labels, regardless of vulnerabilities. As a consequence, using perturbation-based assessments to determine the significance of certain lines of code for the model holds limited interpretive value. Therefore, we can only infer, through the comparison between retrained and previous models, that apart from perturbation-based techniques, other explanation methods can, to some extent, elucidate the model's reasoning process. However, it's important to note the inherent limitations of this evaluation approach.

Second, evaluating the effectiveness of sequence-based explanation methods in locating vulnerabilities remains challenging. We propose TLC and FLC metrics. However, sequence-based detectors provide token-level explanations, making it difficult to directly evaluate TLC and FLC. We approximate token recall from relevant code lines but face challenges due to granularity differences. Determining the most effective method for computing coverage rate in sequence-based explanations remains an open question.

Third, Our research does not involve vulnerability detection methods based on large language models (LLMs). The detectors we employed are primarily based on initial embeddings generated through methods like Word2Vec [34], followed by input into GNN or RNN for binary classification. However, with the introduction of LLMs such as CodeBert [56], significant transformations have emerged in the methodology of code embeddings. These LLMs often exhibit enhanced efficiency when performing code embeddings compared with traditional methods like Word2Vec. Fu et al. [31] demonstrated that LLMs outperform IVDetect. However, we have identified limitations in current binary classification-based detectors, which primarily concentrate on distinctions between vulnerable and normal samples.

6.2 Implications

Limitations of DL-based detectors. During our research, we observed that DL-based detectors primarily learn distinguishing artifacts from the differences between vulnerable and normal samples in the training set to identify potential vulnerability segments, rather than comprehending the inherent semantic essence of vulnerabilities. This, in itself, poses significant challenges for DL-based vulnerability detection.

- **Firstly, acquiring a high-quality vulnerability dataset is not straightforward.** We note that models retrained on a more refined sub-dataset exhibit a greater focus on the VFS compared to the previous models. This indicates a relatively reduced susceptibility to artifacts unrelated to vulnerabilities. However, acquiring such high-quality datasets in real-world scenarios poses considerable challenges, as highlighted by Croft et al. [52], who discovered error rates in labeling vulnerabilities ranging from 20% to 71% within existing real-world datasets.
- **Secondly, facilitating the model to focus more on vulnerability-related lines during the training process is not a straightforward task.** Even when trained on an ideal dataset, models can still be susceptible to perturbations unrelated to vulnerabilities that could flip prediction results (which might also be associated with the nature of DL models). Additionally, when encountering unseen fixing patterns or vulnerable patterns in the training set, models might still make misclassifications. This indicates that the current training methodologies may not necessarily yield an ideal detector.

Insights for future researches. Therefore, performing vulnerability detection based on binary classification may not be an optimal or efficient approach. Note that Kai et al. [57] found that while DL-based methods perform well in code classification and clone detection tasks, their performance in vulnerability detection is unsatisfactory. This indicates a lack of effective

learning of vulnerability semantics by these detectors. We suggest that future work consider dividing vulnerability detection into multiple sub-tasks rather than persisting with methods solely through binary classification tasks. For instance, they could follow the approach proposed by Sun et al. [58], which initially employs LLMs to identify variables and statements within the programs that correlate with vulnerabilities. Subsequently, they can use existing static analysis techniques to verify properties such as accessibility between these variables and statements, thereby identifying vulnerabilities. This approach allows vulnerability detection tools to leverage the latest advancements in DL for code comprehension, particularly the utilization of LLMs. Simultaneously, it mitigates the shortcomings in the reasoning abilities of DL models. Ultimately, this method optimizes the utilization of DL for enhanced and more efficient vulnerability detection.

7 RELATED WORK

7.1 Vulnerability Detection

Malware detection and vulnerability detection are two key focus in software security. Several rule-based detectors have been developed for identifying software vulnerabilities and malicious behavior, such as Clang Static Analyzer [4], Infer [3], Coverity [1], Fortify [5], Rats [2], Checkmarx [6], SVF [7], and MalWuKong [59]. Typically, these detectors rely on human experts to define checking rules using traditional static analysis theories. However, this manual rule definition process can be time-consuming and labor-intensive.

Another research direction focuses on using DL algorithms to identify security vulnerabilities. These DL-based detectors [8–13, 15] can learn vulnerability patterns without the need for manually defining rules, which leads to improved accuracy compared to rule-based approaches. These methods partition programs into multiple code fragments and train a DL model to predict whether those fragments contain vulnerabilities. However, they can only identify vulnerable code fragments without locating detailed statements.

To address the limitation, researchers have proposed various approaches. Li et al. [33] introduce the concept of granularity refinement and present BRNN-vdl, a model that can identify vulnerable tokens within input programs. Fu et al. [31] employ a fine-tuned CodeBert [56] model to predict vulnerable functions and utilize its inner attention mechanism to identify critical sections. Hin et al. [30] tackle vulnerability detection as a node-level classification problem, where they directly predict vulnerable statements. These approaches aim at enhancing the precision and accuracy of identifying specific vulnerability-related statements within code. However, these approaches rely on detection models that require specific models to work, thus lacking generality.

7.2 Explanation Approaches

With the rapid advancements in DL, there has been a growing interest in developing explainability methods for black-box models to provide insights into the decision-making processes of those models. Therefore, these explainability methods can be employed to assist vulnerability detectors in achieving more granular vulnerability detection. Existing explainability techniques can typically be categorized into two distinct groups: instance-level methods and model-level methods. Model-level methods [60, 61] primarily focus on furnishing overarching insights and high-level comprehension for the explication of DL models. In particular, they investigate the correlations between input patterns and specific model behaviors. In contrast, instance-level methods are typically designed to provide explanations for specific inputs and can be further subcategorized into gradient-based methods, surrogate methods, perturbation-based methods, and decomposition methods.

Surrogate methods [27, 62–64] assume that the complexities in the neighboring regions of the input example are relatively low and can be effectively represented by a simpler surrogate model. They usually train interpretable models like random forests [42], to approximate the predictions of the black-box model. While perturbation-based methods [16, 18, 21, 65–69] investigate how perturbations in input features impact the model's output score. By masking out trivial features and focusing on critical ones, these methods aim at identifying the important features contributing to the models' decisions. Gradient-based methods [17, 23] typically compute the importance of input features by analyzing gradients or hidden feature maps. Decomposition methods [22, 25] assess the significance of input features by decomposing the initial model predictions into multiple terms.

7.3 Explainable AI for SE

Despite significant advancements in DL that have greatly benefited automated software engineering, the lack of explainability continues to raise concerns in both the industry and the research community. For instance, questions may arise about why a particular code snippet is predicted to have a vulnerability or why specific comments are associated with a given piece of code. To address these issues, researchers are exploring the application of explainability methods within the field of software engineering.

In the field of software engineering, defect prediction stands out as one of the primary domains where explainability methods find extensive application. For instance, several defect prediction tools [70–72] employ LIME [27] to interpret the predictions of DL models. To address the limitations of LIME, which relies on random perturbations and may yield suboptimal results, Chanathip et al. [73] have introduced PyExplainer to explain defect prediction models. Experimental results have proved its superiority over LIME. Humphrey et al. [74] have achieved defect prediction model interpretation by directly incorporating self-attention mechanisms. In other subfields, Rabin et al. [75] have developed a model-agnostic interpretability method, named Sivand, for method name prediction and variable name misuse prediction models. The key insight lies in the notion that by eliminating extraneous portions of input programs from the prediction process, one can gain a more profound understanding of the essential features involved in model inference. Rabin [76] applied the syntax-aware variant Sivand-Perses to the same code intelligence tasks. They observed that Sivand-Perses is not only faster but also produces more compact sets of critical tokens in the reduced programs. Furthermore, they found that these critical tokens could be employed in generating adversarial examples for as many as 65% of the input programs.

To evaluate the effectiveness of these explainers in the context of vulnerability detection, Ganz et al. [55] conducted a study using various explanation approaches for graph neural networks. They proposed criteria such as stability and descriptiveness to assess the explainers. The study revealed that all evaluated explanation methods exhibited deficiencies in at least two of the criteria. Hu et al. [77] extensively evaluated six popular interpreters on four graph-based vulnerability detectors. They found that these interpreters performed poorly in terms of effectiveness, stability, and robustness. Specifically, instance-independent methods outperformed others in effectiveness, perturbation-based methods showed greater stability, and instance-independent approaches provided more consistent results for similar vulnerabilities in terms of robustness. Compared to the previous two works, our research uncovers additional concerns within DL-based detectors. Specifically, we identify that these detectors, despite achieving accurate detection performance, may still learn features unrelated to vulnerabilities. Moreover, they are susceptible to being misled by irrelevant perturbations, leading to a reversal in predictions, thereby influencing the effectiveness of explanation methods.

These explanation methods serve as valuable tools for gaining insights into the decision-making process of black-box models, and their application in vulnerability detection is an area of active research.

8 CONCLUSION

In this study, we systematically evaluate the challenges in locating vulnerabilities with DL-based detectors and explanation approaches. Firstly, we evaluate the effectiveness of the explanation approaches using two metrics: fidelity and LC. We find that the results of fidelity and LC show little correlation. Meanwhile, these explanation methods show limited effectiveness in locating vulnerable lines. Secondly, subsequent experiments with DL-based detectors suggest that they might learn artifacts unrelated to the vulnerabilities, as retraining models on a balanced dataset can indeed improve model-related explainers' performance regarding FLC. This phenomenon may stem from vulnerability-unrelated distinctions between vulnerable and normal samples, thereby diminishing the efficacy of explanation approaches in identifying vulnerable lines. Furthermore, even when trained on a balanced dataset, models remain susceptible to vulnerability-unrelated perturbations, undermining the reliability of fidelity calculations or explanation approaches based on perturbation. Based on our findings, employing binary classification models directly for vulnerability detection may not be effective. We suggest that future work endeavors decompose vulnerability detection into multiple subtasks and utilize deep learning techniques to improve efficiency.

ACKNOWLEDGMENTS

This work was partially supported by the National Natural Science Foundation of China (No. 62141208), the National Natural Science Foundation of China (grant No. 62072046), the Key R&D Program of Hubei Province (2023BAB017, 2023BAB079), HUST CSE-HongXin Joint Institute for Cyber Security, and HUST CSE-FiberHome Joint Institute for Cyber Security, and the Cyber Security Agency of Singapore under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN). Any opinions, findings, conclusions, or recommendations expressed in this material are those of the author(s) and do not reflect the views of the respective funding agencies.

REFERENCES

[1] Coverity. 2020. Retrieved from https://scan.coverity.com/
[2] RATS. 2014. Retrieved from https://code.google.com/archive/p/rough-auditing-tool-for-security/
[3] Infer. 2020. Retrieved from https://fbinfer.com/
[4] Clang static analyzer. 2020. Retrieved from https://clang-analyzer.llvm.org/scan-build.html
[5] HP Fortify. 2020. Retrieved from https://www.hpfod.com/
[6] Checkmarx. 2020. Retrieved from https://www.checkmarx.com/
[7] Yulei Sui and Jingling Xue. 2016. SVF: Interprocedural static value-flow analysis in LLVM. In *CC'16*. 265–266. Retrieved from https://github.com/unsw-corg/SVF.
[8] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 10197–10207.
[9] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296.
[10] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2021. Vulnerability detection with fine-grained interpretations. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 292–303.
[11] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, and Yulei Sui. 2021. DeepWukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Trans. Softw. Eng. Methodol.* 30, 3, Article 38 (2021), 33 pages. DOI: http://dx.doi.org/10.1145/3436877

[12] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2021. SySeVR: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* (2021), 1–1. DOI: http://dx.doi.org/10.1109/TDSC.2021.3051525

[13] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*. The Internet Society.

[14] Yueming Wu, Deqing Zou, Shihan Dou, Wei Yang, Duo Xu, and Hai Jin. 2022. Vulcnn: An image-inspired scalable vulnerability detection system. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE'22)*, 2365–2376.

[15] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated vulnerability detection in source code using deep representation learning. In *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA'18)*, 757–762.

[16] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.). Vol. 32. Curran Associates, Inc.

[17] Aditya Chattopadhay, Anirban Sarkar, Prantik Howlader, and Vineeth N. Balasubramanian. 2018. Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks. In *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 839–847.

[18] Deqing Zou, Yawei Zhu, Shouhuai Xu, Zhen Li, Hai Jin, and Hengkai Ye. 2021. Interpreting deep learning-based vulnerability detector predictions based on heuristic searching. *ACM Transactions on Software Engineering and Methodology* 30, 2 (2021), 1–31.

[19] Kumar Abhishek and Deeksha Kamath. 2022. Attribution-based XAI methods in computer vision: A Review. arXiv:2211.14736. Retrieved from https://arxiv.org/abs/2211.14736

[20] Andreas Madsen, Siva Reddy, and Sarath Chandar. 2022. Post-hoc interpretability for neural nlp: A survey. *ACM Computing Surveys* 55, 8 (2022), 1–42.

[21] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS'20)*. Vol. 33, Curran Associates Inc., Red Hook, NY, 19620–19631.

[22] Thomas Schnake, Oliver Eberle, Jonas Lederer, Shinichi Nakajima, Kristof T. Schütt, Klaus-Robert Müller, and Grégoire Montavon. 2022. Higher-order explanations of graph neural networks via relevant walks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 11 (2022), 7581–7596.

[23] Phillip E. Pope, Soheil Kolouri, Mohammad Rostami, Charles E. Martin, and Heiko Hoffmann. 2019. Explainability methods for graph convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10772–10781.

[24] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. 2017. Learning important features through propagating activation differences. In *International Conference on Machine Learning*. PMLR, 3145–3153.

[25] Leila Arras, Grégoire Montavon, Klaus-Robert Müller, and Wojciech Samek. 2017. Explaining recurrent neural network predictions in sentiment analysis. In *Proceedings of the 8th Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis*. Association for Computational Linguistics, Copenhagen, Denmark, 159–168. DOI: http://dx.doi.org/10.18653/v1/W17-5221

[26] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*, Curran Associates Inc., Red Hook, NY, 4768–4777.

[27] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. " Why should i trust you?" explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1135–1144.

[28] Xiao Cheng, Guanqin Zhang, Haoyu Wang, and Yulei Sui. 2022. Path-sensitive code embedding via contrastive learning for software vulnerability detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 519–531.

[29] Huanting Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, and Zheng Wang. 2021. Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Transactions on Information Forensics and Security* 16 (2021), 1943–1958.

[30] David Hin, Andrey Kan, Huaming Chen, and M. Ali Babar. 2022. LineVD: statement-level vulnerability detection using graph neural networks. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 596–607.

[31] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 608–620.

[32] Weining Zheng, Yuan Jiang, and Xiaohong Su. 2021. Vu1SPG: Vulnerability detection based on slice property graph representation learning. In *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 457–467.

[33] Zhen Li, Deqing Zou, Shouhuai Xu, Zhaoxuan Chen, Yawei Zhu, and Hai Jin. 2021. Vuldeelocator: A deep learning-based fine-grained vulnerability detector. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2021), 2821–2837.

[34] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, C. J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Q. Weinberger (Eds.). Vol. 26, Curran Associates, Inc., 3111–3119.

[35] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard S. Zemel. 2016. Gated graph sequence neural networks. In *4th International Conference on Learning Representations (ICLR'16)*, San Juan and Puerto Rico (Eds.).

[36] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE Symposium on Security and Privacy*. IEEE, 590–604.

[37] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 1532–1543.

[38] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. On the properties of neural machine translation: Encoder-decoder approaches. In *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, Association for Computational Linguistics, Doha, 103–111.

[39] Thomas N. Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings, ICLR'17*, 2017.

[40] Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International Conference on Machine Learning*. PMLR, 1188–1196.

[41] A. Graves and J. Schmidhuber. 2005. Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Networks* 18, 5–6 (2005), 602–610.

[42] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32.

[43] Xiao Cheng, Xu Nie, Ningke Li, Haoyu Wang, Zheng Zheng, and Yulei Sui. 2022. How about bug-triggering paths?-understanding and characterizing learning-based vulnerability detectors. *IEEE Transactions on Dependable and Secure Computing*, 1–18.

[44] Kai Sheng Tai, Richard Socher, and Christopher D. Manning. 2015. Improved semantic representations from tree-structured long short-term memory networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing, ACL 2015, July 26-31, 2015, Beijing, China, Volume 1: Long Papers*. The Association for Computer Linguistics, 1556–1566. DOI: http://dx.doi.org/10.3115/v1/p15-1150

[45] Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. 2023. Explainability in graph neural networks: A taxonomic survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45, 5 (2023), 5782–5799.

[46] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision*. 618–626.

[47] W. James Murdoch, Chandan Singh, Karl Kumbier, Reza Abbasi-Asl, and Bin Yu. 2019. Definitions, methods, and applications in interpretable machine learning. *Proceedings of the National Academy of Sciences* 116, 44 (2019), 22071–22080.

[48] Software Assurance Reference Dataset. 2017. Retrieved from https://samate.nist.gov/SARD/index.php

[49] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ code vulnerability dataset with code changes and CVE summaries. In *MSR'20: 17th International Conference on Mining Software Repositories*.

[50] Marko Robnik-Šikonja and Marko Bohanec. 2018. Perturbation-based explanations of prediction models. In *Human and Machine Learning*. Springer, 159–175.

[51] Yunhui Zheng, Saurabh Pujar, Burn Lewis, Luca Buratti, Edward Epstein, Bo Yang, Jim Laredo, Alessandro Morari, and Zhong Su. 2021. D2A: A dataset built for AI-based vulnerability detection methods using differential analysis. In *Proceedings of the ACM/IEEE 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP'21)*. Association for Computing Machinery, New York, NY, USA.

[52] Roland Croft, M. Ali Babar, and M. Mehdi Kholoosi. 2023. Data quality for software vulnerability datasets. In *IEEE/ACM 45th International Conference on Software Engineering (ICSE'23)*, 121–133.

[53] Xu Nie, Ningke Li, Kailong Wang, Shangguang Wang, Xiapu Luo, and Haoyu Wang. 2023. Understanding and tackling label errors in deep learning-based vulnerability detection (experience paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 52–63.

[54] Joern, 2014. https://github.com/joernio/joern
[55] Tom Ganz, Martin Härterich, Alexander Warnecke, and Konrad Rieck. 2021. Explaining graph neural networks for vulnerability discovery. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security*. 145–156.
[56] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL)*, Trevor Cohn, Yulan He, and Yang Liu (Eds.), Vol. EMNLP 2020. Association for Computational Linguistics, 1536–1547. DOI: http://dx.doi.org/10.18653/v1/2020.findings-emnlp.139
[57] Siow Jing Kai, Liu Shangqing, Xie Xiaofei, Meng Guozhu, and Liu Yang. 2022. Learning program semantics with code representations: An empirical study. In *Proceedings of the 29th IEEE International Conference on Software Analysis, Evolution and Reengineering*.
[58] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2023. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. arXiv preprint arXiv:2308.03314.
[59] Ningke Li, Shenao Wang, Mingxi Feng, Kailong Wang, Meizhen Wang, and Haoyu Wang. 2023. MalWuKong: Towards fast, accurate, and multilingual detection of malicious code poisoning in OSS supply chains. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1993–2005.
[60] Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. 2017. Feature visualization. *Distill* 2, 11 (2017), e7.
[61] Hao Yuan, Jiliang Tang, Xia Hu, and Shuiwang Ji. 2020. Xgnn: Towards model-level explanations of graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 430–438.
[62] Qiang Huang, Makoto Yamada, Yuan Tian, Dinesh Singh, and Yi Chang. 2023. Graphlime: Local interpretable model explanations for graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* 35, 7 (2023), 6968–6972.
[63] Minh N. Vu and My T. Thai. 2020. Pgm-explainer: probabilistic graphical model explanations for graph neural networks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS'20)*, Curran Associates Inc., Red Hook, NY.
[64] Yue Zhang, David Defazio, and Arti Ramesh. 2021. Relex: A model-agnostic relational model explainer. In *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*. 1042–1049.
[65] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On explainability of graph neural networks via subgraph explorations. In *International Conference on Machine Learning*. PMLR, 12241–12252.
[66] Thorben Funke, Megha Khosla, and Avishek Anand. 2021. Hard masking for explaining graph neural networks. In *Proceedings of the 2021 International Conference on Learning Representations (ICLR'21)*.
[67] Xiang Wang, Yingxin Wu, An Zhang, Xiangnan He, and Tat-seng Chua. 2020. Causal screening to interpret graph neural networks. (2020).
[68] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. 2021. Interpreting graph neural networks for NLP with differentiable edge masking. In *9th International Conference on Learning Representations (ICLR '21, Virtual Event, Austria, May 3-7, 2021)*, OpenReview.net, 2021.
[69] Baijun Cheng, Kailong Wang, Cuiyun Gao, Xiapu Luo, Yulei Sui, Li Li, Yao Guo, Xiangqun Chen, and Haoyu Wang. 2024. The vulnerability is in the details: Locating fine-grained information of vulnerable code identified by graph-based detectors. arXiv preprint arXiv:2401.02737.
[70] Supatsara Wattanakriengkrai, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Hideaki Hata, and Kenichi Matsumoto. 2020. Predicting defective lines using a model-agnostic technique. *IEEE Transactions on Software Engineering* 48, 5 (2020), 1480–1496.
[71] Chanathip Pornprasit and Chakkrit Kla Tantithamthavorn. 2021. Jitline: A simpler, better, faster, finer-grained just-in-time defect prediction. In *IEEE/ACM 18th International Conference on Mining Software Repositories (MSR'21)*, IEEE, 369–379.
[72] Wei Zheng, Tianren Shen, and Xiang Chen. 2021. Just-in-time defect prediction technology based on interpretability technology. In *2021 8th International Conference on Dependable Systems and Their Applications (DSA)*. IEEE, 78–89.
[73] Chanathip Pornprasit, Chakkrit Tantithamthavorn, Jirayus Jiarpakdee, Michael Fu, and Patanamon Thongtanunam. 2021. Pyexplainer: Explaining the predictions of just-in-time defect models. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 407–418.
[74] Jack Humphreys and Hoa Khanh Dam. 2019. An explainable deep model for defect prediction. In *2019 IEEE/ACM 7th International Workshop on Realizing Artificial Intelligence Synergies in Software Engineering (RAISE)*. IEEE, 49–55.
[75] Md Rafiqul Islam Rabin, Vincent J. Hellendoorn, and Mohammad Amin Alipour. 2021. Understanding neural code intelligence through program simplification. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 441–452.

[76] Md Rafiqul Islam Rabin, Aftab Hussain, and Mohammad Amin Alipour. 2022. Syntax-guided program reduction for understanding neural code intelligence models. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 70–79.

[77] Yutao Hu, Suyuan Wang, Wenke Li, Junru Peng, Yueming Wu, Deqing Zou, and Hai Jin. 2023. Interpreters for GNN-based vulnerability detection: Are we there yet? In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1407–1419.